Maharaja Education Trust (R), Mysuru

Maharaja Institute of Technology Mysore

Belawadi, Sriranga Pattana Taluk, Mandya – 571 477

Approved by AICTE, New Delhi,
Affiliated to VTU, Belagavi & Recognized by Government of Karnataka

Lab Manual on

INTERNET OF THINGS LABORATORY

(M24MCAL308)

Prepared by,

Department of Master of Computer Applications

Vision

"To be recognized as a premier technical and management institution promoting extensive education fostering research, innovation and entrepreneurial attitude"

Mission

- To empower students with indispensable knowledge through dedicated teaching and collaborative learning.
- To advance extensive research in science, engineering and management disciplines.
- To facilitate entrepreneurial skills through effective institute - industry collaboration and interaction with alumni.
- To instill the need to uphold ethics in every aspect.
- To mould holistic individuals capable of contributing to the advancement of the society.

Maharaja Institute of Technology Mysore

Department of MCA

Vision

To be the premier source that provides a transformative education to create computer professionals, competent entrepreneurs & technocrats to meet the global challenges.

Mission

- To facilitate students to nurture skills to practice their professions competently to meet the ever-changing needs of society & environment
- To develop the potential of human resources with new ideas, research to meet the requirements of cutting edge technology

Program Outcomes

PO1 (Foundation Knowledge): Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.

PO2 (Problem Analysis): Identify, review, formulate and analyze problems for primarily focusing on customer requirements using critical thinking frameworks.

PO3 (Development of Solutions): Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.

PO4 (Modern Tool Usage): Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO5 (Individual and Teamwork): Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.

PO6 (Project Management and Finance): Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.

PO7 (Ethics): Commit to professional ethics in managing software projects with financial aspects. Learn to use new technologies for cyber security and insulate customers from malware

PO8 (Life-long learning): Change management skills and the ability to learn, keep up with Contemporary technologies and ways of working.

Maharaja Institute of Technology Mysore

Department of MCA

Course Overview

Subject: Internet of Things Lab **Subject Code: M24MCAL308**

This course introduces students to the fundamental concepts, hardware components, and software tools used in the Internet of Things (IoT). It provides a practical, hands-on approach to understanding how everyday devices can be connected, monitored, and controlled over the Internet. Students will work with popular development platforms such as **Arduino** and **Raspberry Pi**, learning their architecture, pin configurations, and programming methods.

The course focuses on interfacing sensors and actuators including LEDs, push buttons, RGB LEDs, potentiometers, LDRs, temperature sensors (LM35), servo motors, ultrasonic sensors, relays, and buzzers. Through step-by-step experiments, students will design basic IoT applications such as LED control, traffic light simulation, distance measurement, environmental monitoring, and automated switching based on sensor data.

Students will also explore mobile-based IoT control using the **Blynk application**, enabling wireless operation of devices like LEDs, relays, and buzzers. Additionally, the course introduces cloud platforms such as **ThingSpeak**, where learners upload and analyze sensor data like temperature, humidity, and ambient light levels.

Advanced activities include converting a USB webcam into a fully functional IP camera and building a Raspberry Pi–based weather reporting system. By the end of the course, students gain confidence in developing simple IoT prototypes, integrating hardware and software, and understanding real-world IoT applications.

Course Objectives

The objectives of this course is to make students to learn-

- To understand the fundamentals of IoT hardware platforms such as Arduino and Raspberry Pi.
- To learn the interfacing of various sensors and actuators with IoT devices for real-time applications.
- To develop the ability to program microcontrollers and microcomputers for controlling and automating systems.
- To design and implement IoT-based automation and monitoring solutions for practical scenarios.
- To acquire skills in data acquisition, processing, and display using IoT components.
- To build and test IoT projects addressing real-world challenges such as smart farming, health monitoring, and security systems.

Course Outcomes

CO's	DESCRIPTION OF THE OUTCOMES
M24MCAL308.1	Apply programming constructs in Python and C to develop logic, capture sensor data, and implement embedded processing for IoT applications.
M24MCAL308.2	Interface sensors and actuators with Arduino or Raspberry Pi using C or Python for real time IoT solutions.
M24MCAL308.3	Design and execute small-scale IoT projects by integrating programming, hardware components, and network communication.
M24MCAL308.4	Present technical ideas and project outcomes effectively using professional written documentation and oral skills.

Syllabus

Internet of Things Laboratory [As per Choice Based Credit System (CBCS) scheme] SEMESTER –III			
Course Code	**M24MCAL308**	CIE Marks	**50**
Number of Lecture Hours / Week	**01**	SEE Marks	**50**
Total Number of Lecture Hours	**32**	Exam Hours	**03**
CREDITS-01			

Laboratory Experiments	
1.	Study the fundamentals of IOT software's and components Familiarization with the detailed concepts of Arduino / Raspberry Pi
2.	I. Interfacing the Light Emitting Diode (LED) . II. Interfacing the RGB LED with the Arduino. III. Traffic Signal light Simulation. **IV.** Controlling the LED blink rate with the potentiometer interfacing with Arduino.
3.	I. Interfacing of temperature sensor LM35 with Arduino. **II.** Interfacing Servo Motor with the Arduino.
4.	I. Interfacing of the Active Buzzer with Arduino. **II.** Interfacing of the Relay with Arduino.
5.	I. Calculate the distance to an object with the help of an ultrasonic sensor and display it on an LCD. **II.** Automatic Water Tank Level Controller Using Arduino
6.	I. Smoke & Gas Leak Detector Using Arduino. **II.** ii. Digital Stopwatch Using Arduino
7.	Password Based Door Lock System Using Arduino
8.	IoT Based Smart Farming System.
9.	Industrial Automation System Using Arduino.
10.	Patient Health Monitoring System Using Arduino
11.	Interfacing the regular USB webcam with the device and capture the image
12.	Designing a weather forecasting system using Raspberry Pi.

Index

Subject: Internet of Things Laboratory **Subject Code: M24MCAL308**

S/L	Contents	Page No.
1	Study the fundamentals of IOT software's and components Familiarization with the detailed concepts of Arduino / Raspberry Pi	1-5
2	I. Interfacing the Light Emitting Diode (LED) . II. Interfacing the RGB LED with the Arduino. III. Traffic Signal light Simulation. **IV.** Controlling the LED blink rate with the potentiometer interfacing with Arduino.	6-16
3	I. Interfacing of temperature sensor LM35 with Arduino. **II.** Interfacing Servo Motor with the Arduino.	17-20
4	I. Interfacing of the Active Buzzer with Arduino. II. Interfacing of the Relay with Arduino.	21-24
5	**I.** Calculate the distance to an object with the help of an ultrasonic sensor and display it on an LCD.	25-26
6	**I.** Smoke & Gas Leak Detector Using Arduino. **II.** Digital Stopwatch Using Arduino	27-32
7	Password Based Door Lock System Using Arduino	33-39
8	IoT Based Smart Farming System.	40-43
9	Patient Health Monitoring System Using Arduino	44-47
10	Interfacing the regular USB webcam with the device and capture the image	48-49
11	Designing a weather forecasting system using Raspberry Pi.	50-53
12	Viva – voce	54-63

1. Study the fundamentals of IOT software's and components

To study the fundamentals of the Internet of Things (IoT), it is essential to understand both the hardware components that interact with the physical world and the software layers that process and manage the resulting data.

1. Core Hardware Components

IoT hardware acts as the interface between the physical and digital worlds.

Sensors (Inputs): These devices detect changes in the environment, such as temperature, humidity, motion, light, or pressure, and convert them into digital signals.

Actuators (Outputs): These receive commands to perform physical actions, such as opening a valve, turning on a light, or moving a motor.

Microcontrollers (MCUs) & Processors: The "brain" of the device (e.g., Arduino, Raspberry Pi, NodeMCU) that processes local logic and manages data flow.

Communication Modules: Hardware components like Wi-Fi chips, Bluetooth antennas, or cellular modems that enable the device to connect to a network.

2. Software Fundamentals

IoT software architecture is typically divided into four main layers that ensure data moves seamlessly from the device to the user.

Device Layer (Firmware): The low-level software running directly on the hardware to manage resources, read sensors, and execute basic commands.

Connectivity Layer (Protocols): Protocols that define how data is transmitted.

Short Range: Bluetooth Low Energy (BLE), Zigbee, Z-Wave.

Long Range: LoRaWAN, NB-IoT, Cellular (4G/5G).

Application Protocols: MTT (lightweight, messaging), CoAP (for constrained devices), and HTTP.

Management Layer (Middleware): Software that bridges the gap between hardware and applications. It handles device discovery, data filtering, and security updates.

Application Layer: The software that analyzes data (often using AI or Big Data) and provides the User Interface (UI) through mobile apps or web dashboards.

3. IoT Software (Arduino & Raspberry Pi)

a. *Operating Systems & Development Environment*
   - Arduino – Uses Arduino IDE, which supports C/C++. No operating system, just firmware.
   - Raspberry Pi – Runs Raspberry Pi OS (Linux-based) and supports programming in Python, C++, Java, etc.

b. *Communication Protocols (Used for data transfer)*

- Wired – UART (Serial), SPI, I2C (for sensor communication).
- Wireless – Wi-Fi, Bluetooth, Zigbee, LoRa, MTT (for internet/cloud).

c. IoT Programming Languages

- Arduino – C/C++ (written in Arduino IDE).
- Raspberry Pi – Python (mostly), also supports C, Java, and JavaScript.

d. Cloud & Data Storage

- Arduino – Sends data to the cloud using Wi-Fi/GSM modules (e.g., Firebase, ThingSpeak).
- Raspberry Pi – Stores data locally or in the cloud (e.g., AWS IoT, Google Cloud IoT).

4. Different types of Sensors

- **Temperature Sensors**: Measure heat or temperature changes in the environment or objects.
- **Image Sensors**: Capture visual data for cameras and computer vision applications.
- **Gyro Sensors (Gyroscope)**: Detect angular velocity and orientation of objects.
- **Obstacle Sensors**: Identify the presence of obstacles to avoid collisions.
- **RF Sensors:** Use radio freuency signals for detection, tracking, and communication.
- **IR Sensors (Infrared):** Detect heat signatures or motion using infrared light.
- **M-02/05 Gas Sensors**: Sense the presence and concentration of gases like CO, methane, or smoke.
- **LDR Sensor (Light Dependent Resistor):** Measure light intensity and brightness levels.
- **Ultrasonic Distance Sensor**: Calculate distance by using ultrasonic sound waves.

5. History of IOT

1982: Vending Machine: First IoT concept; reported inventory status remotely.

1990: Toaster: First internet-connected appliance; remote control of devices.

1999:IoT Term Coined: Kevin Ashton introduced "Internet of Things."

2000: LG Smart Fridge: Remote monitoring of fridge contents; IoT in daily life.

2004: Smart Watch: Wearables with fitness tracking & notifications.

2007: iPhone: Smartphones became IoT hubs via apps and connectivity.

2009: Cars: IoT enabled diagnostics , performance monitoring.

2011: Smart TV: Internet enabled entertainment & apps.

2013: Google Lens: Object recognition linking physical world to digital info.

2014: Amazon Echo: Voice controlled smart home via Alexa.

2015: Tesla Autopilot: Semiautonomous driving with IoT sensors/software.

Advantages of IoT

Improved efficiency and automation of tasks.

Increased convenience and accessibility of information.

Better monitoring and control of devices and systems.

Disadvantages of IoT

Potential for hacking and data breaches.

Collection and misuse of personal data.

Significant initial investment reuired.

2. Familiarization with the detailed concepts of Arduino / Raspberry Pi

Arduino Board

Arduino is a small electronic board used to build IoT and automation projects. It has a Microcontroller (a tiny computer) that can read sensors, control lights, motors, and send data.

Common Arduino Boards

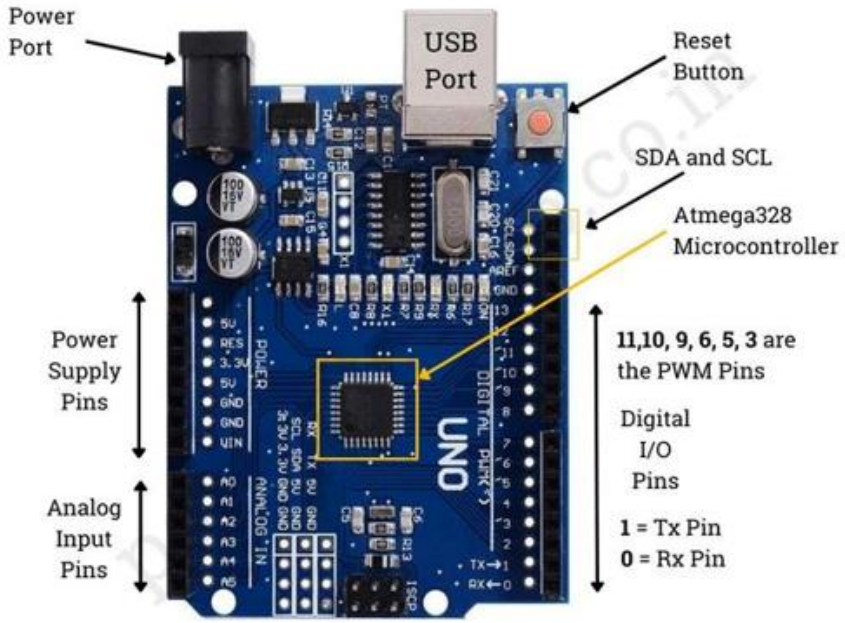
- Arduino Uno – Best for beginners, has 14 digital and 6 analog pins.
- Arduino Mega – More powerful, with 54 digital and 16 analog pins.
- Arduino Nano – A smaller version of Uno, useful for compact projects.
- Arduino Leonardo – Has built-in USB support (acts like a keyboard/mouse).
- Arduino Due – Faster, with a 32-bit microcontroller.

Key Parts of an Arduino Board

- Microcontroller – The brain (e.g., ATmega328P in Uno).
- Digital & Analog Pins – Connect sensors and actuators.
- Power Port – Runs on USB or battery (5V/9V).
- USB Port – Connects to a computer for programming.
- Reset Button – Restarts the board.

How to Use Arduino?

- Write code using the Arduino IDE (C/C++ based).
- Upload code via USB to the board.
- Connect sensors & actuators (LEDs, motors, etc.).
- Run the project!



Arduino UNO Pinout

Raspberry Pi

Raspberry Pi is a small computer (not just a microcontroller like Arduino) that can run an operating system, connect to the internet, and control hardware like sensors and motors. It is great for IoT, robotics, and AI projects.

Common Raspberry Pi Models

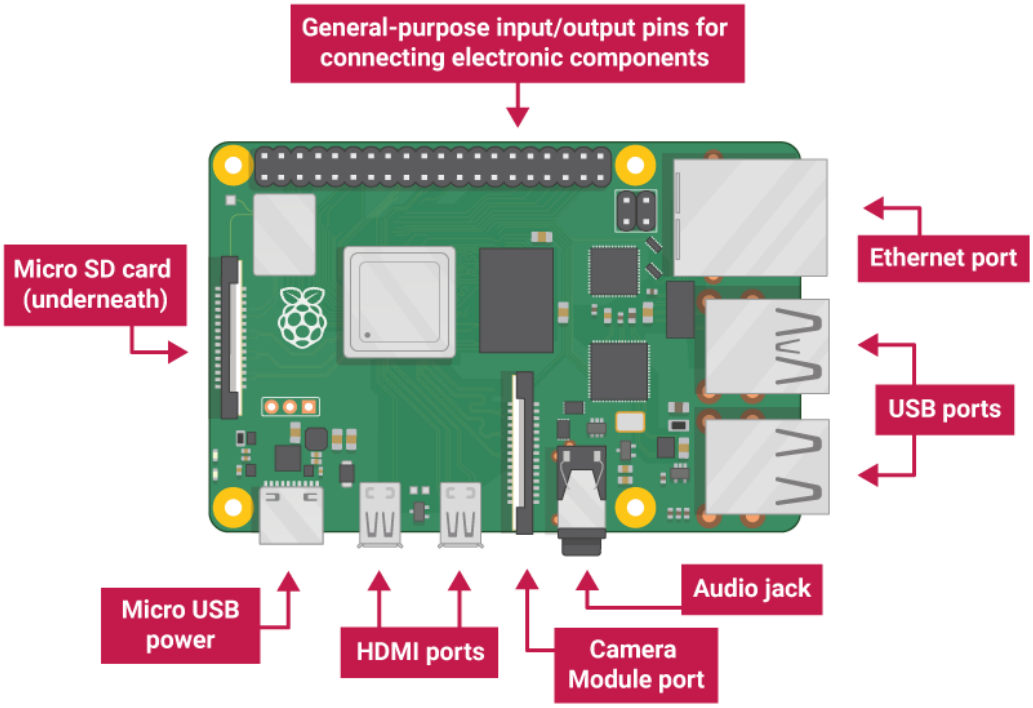
- Raspberry Pi 4 – Most powerful, supports 4K video, Wi-Fi, Bluetooth.
- Raspberry Pi 3B+ – Good for IoT projects, has built-in Wi-Fi and Bluetooth.
- Raspberry Pi Zero – Smallest, best for low-power IoT applications.
- Raspberry Pi Pico – A microcontroller (not a full computer) like Arduino.

Key Parts of Raspberry Pi

- CPU (Processor) – Works like a mini-computer.
- RAM – Temporary memory for running programs.
- GPIO Pins – Connect sensors, LEDs, and motors.
- USB & HDMI Ports – Connect a keyboard, mouse, and display.
- MicroSD Card Slot – Stores the operating system (e.g., Raspberry Pi OS).
- Wi-Fi & Bluetooth – Available in most models for wireless connections.

Key Parts of Raspberry Pi

- Install OS – Use Raspberry Pi OS on a microSD card.
- Connect to Display – Use HDMI to connect a monitor.
- Attach Keyboard & Mouse – Just like a regular computer.
- Write Code – Use Python, C, or Java for projects.
- Control Electronics – Use GPIO pins to connect sensors and actuators.



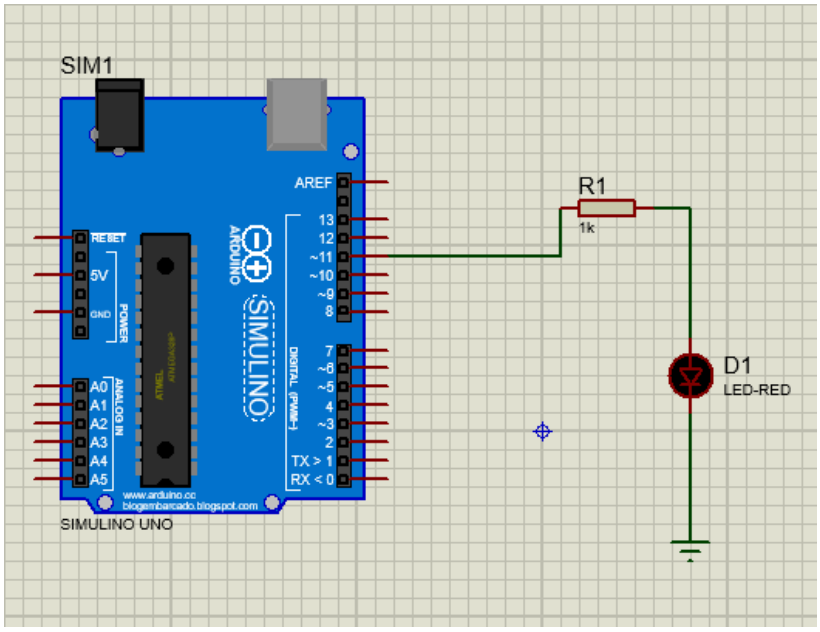
3. i. Interfacing the Light Emitting Diode (LED)
ii. Interfacing the RGB LED with the Arduino
iii. Traffic Signal light Simulation

i. Interfacing the Light Emitting Diode (LED)

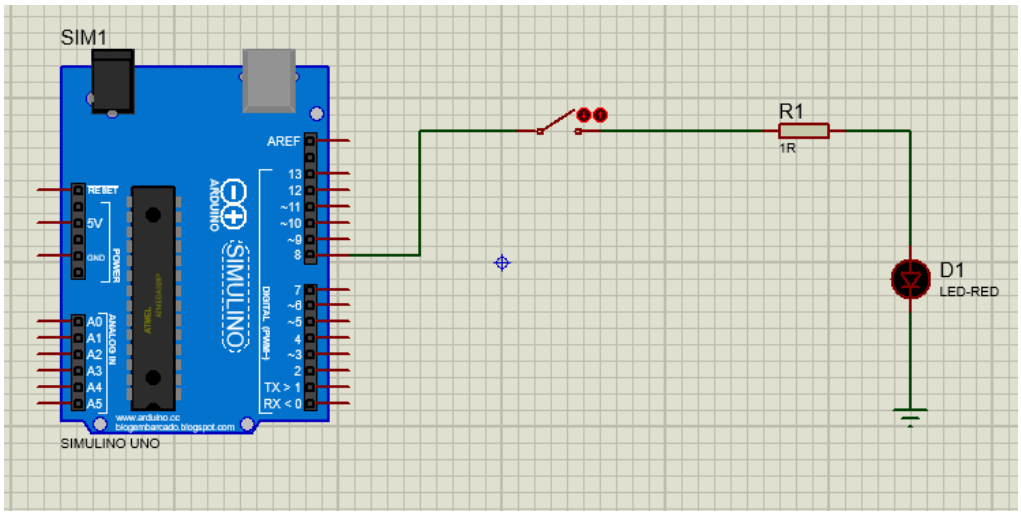
Reuirements

1. Arduino Uno
2. Led -1
3. Resistor – 330ohm
4. Connecting wires

Circuit Connection



Or



Program

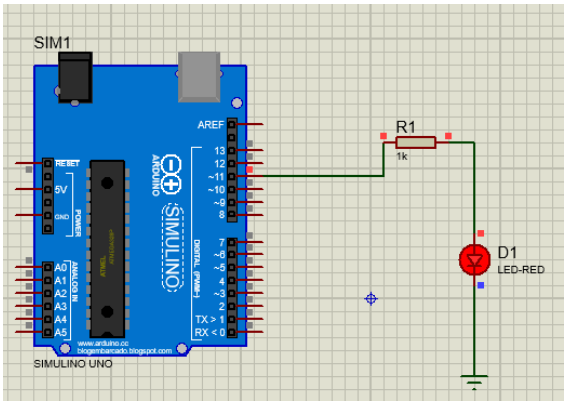
```
void setup() {
```

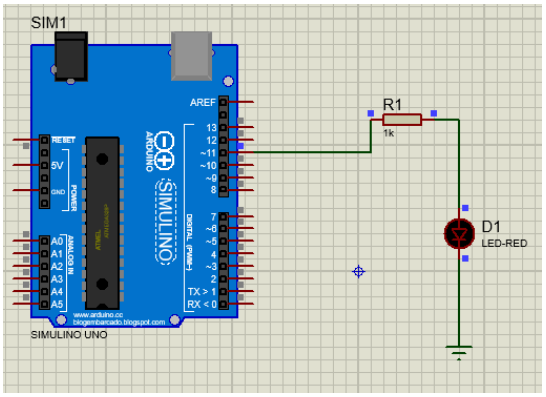
```
pinMode(11, OUTPUT);
}
void loop() {
digitalWrite(11, HIGH);
delay(1000);
digitalWrite(11, LOW);
delay(1000);
}
```

OR

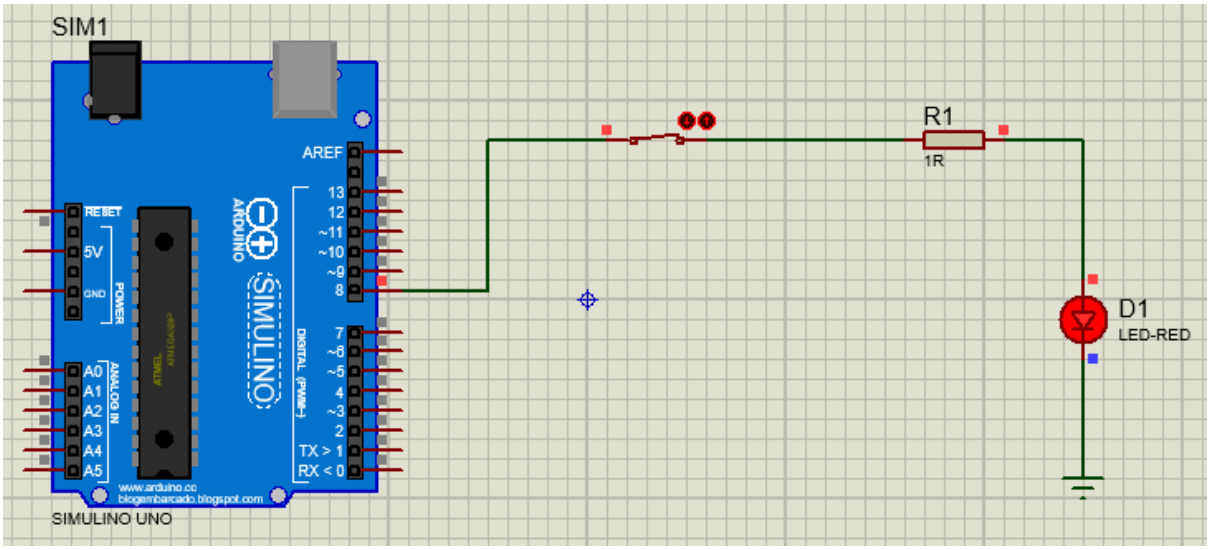
```
void setup() {
 pinMode(8, OUTPUT);
}
void loop() {
   digitalWrite(8, HIGH);
}
```

OUTPUT





OR

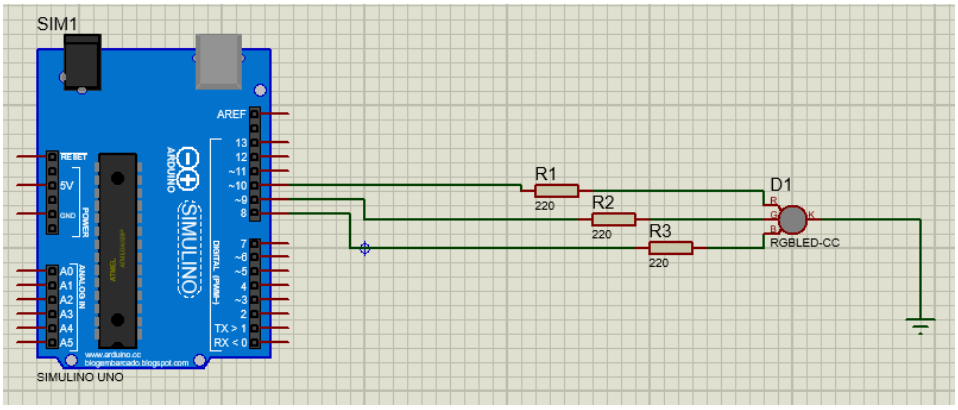


ii. Interfacing the RGB LED with the Arduino

Reuirements

1. Arduino Uno
2. RGBLed -1
3. Resistor – 220 ohm – 3Nos
4. Connecting wires

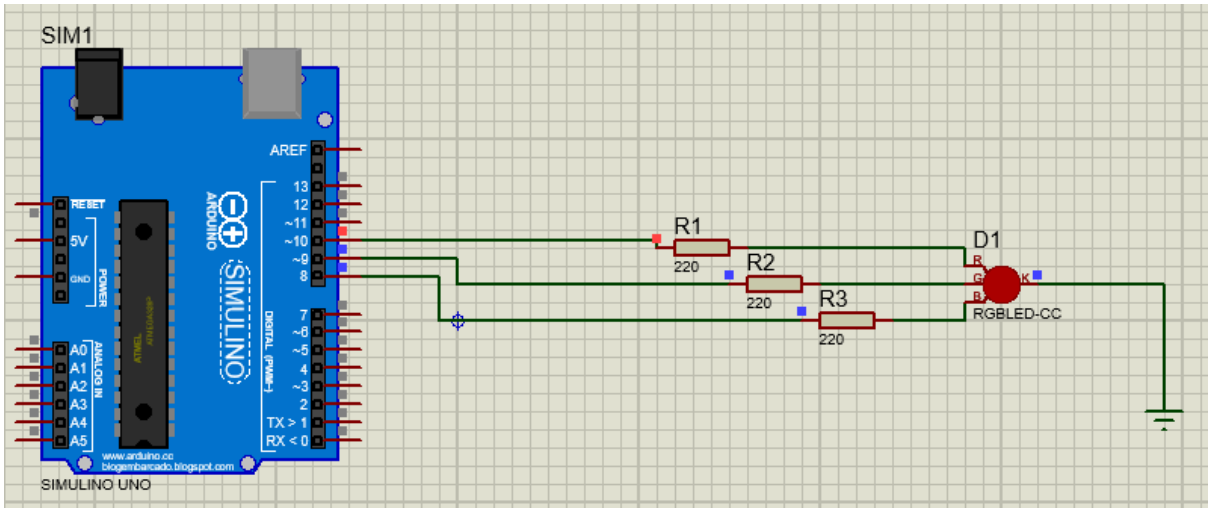
Circuit Connection

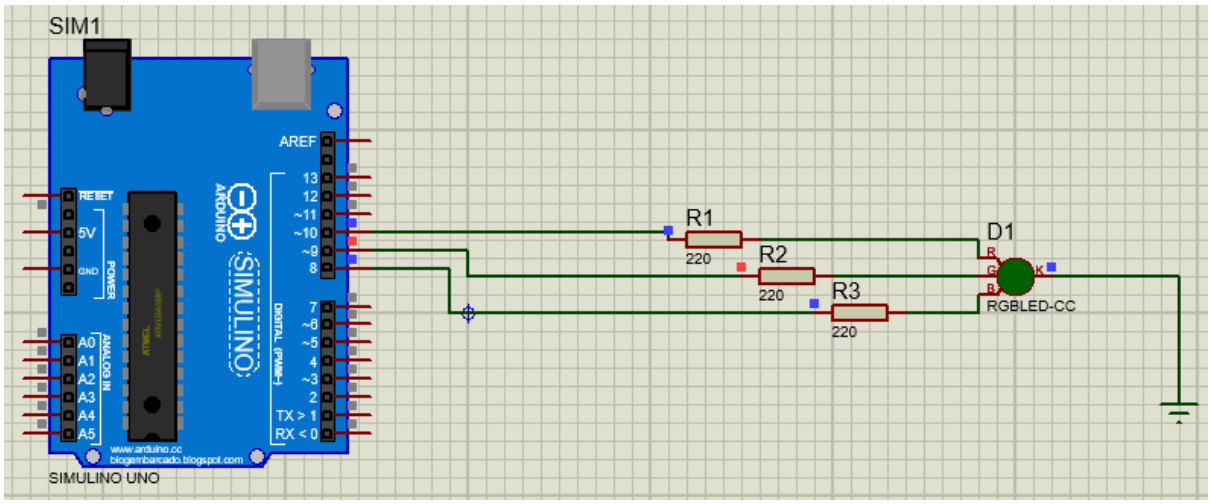


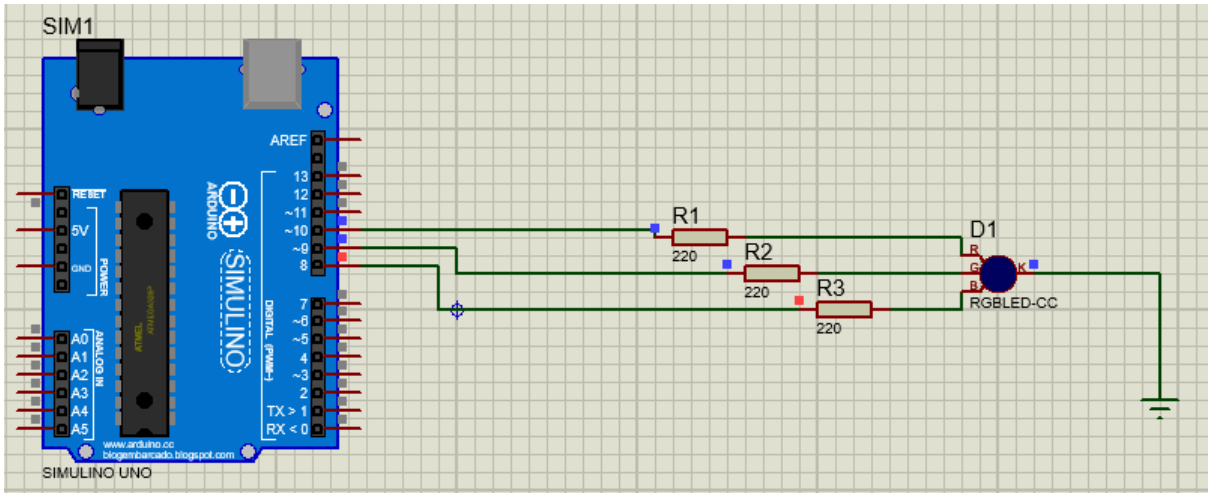
Program

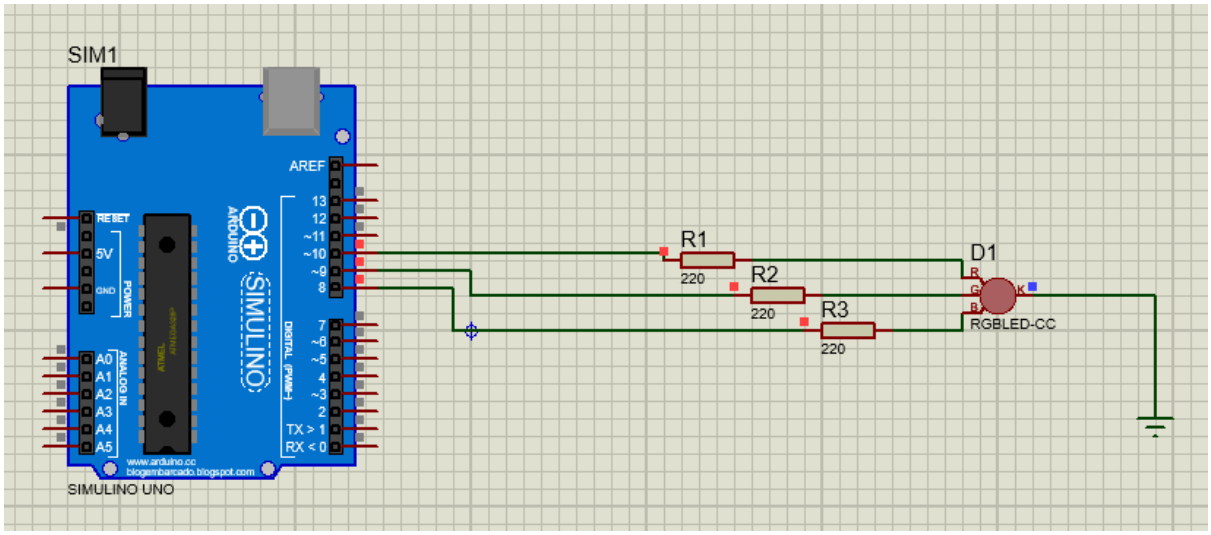
```
int redPin= 10;
int greenPin = 9;
int bluePin = 8;
void setup() {
        pinMode(redPin, OUTPUT);
        pinMode(greenPin, OUTPUT);
        pinMode(bluePin, OUTPUT);
}
void loop() {
        setColor(255, 0, 0); // Red Color
        delay(1000);
        setColor(0, 255, 0); // Green Color
        delay(1000);
        setColor(0, 0, 255); // Blue Color
        delay(1000);
        setColor(255, 255, 255); // White Color
        delay(1000);
        setColor(170, 0, 255); // Purple Color
        delay(1000);
}
void setColor(int redValue, int greenValue, int blueValue) {
        analogWrite(redPin, redValue);
        analogWrite(greenPin, greenValue);
        analogWrite(bluePin, blueValue);
}
```

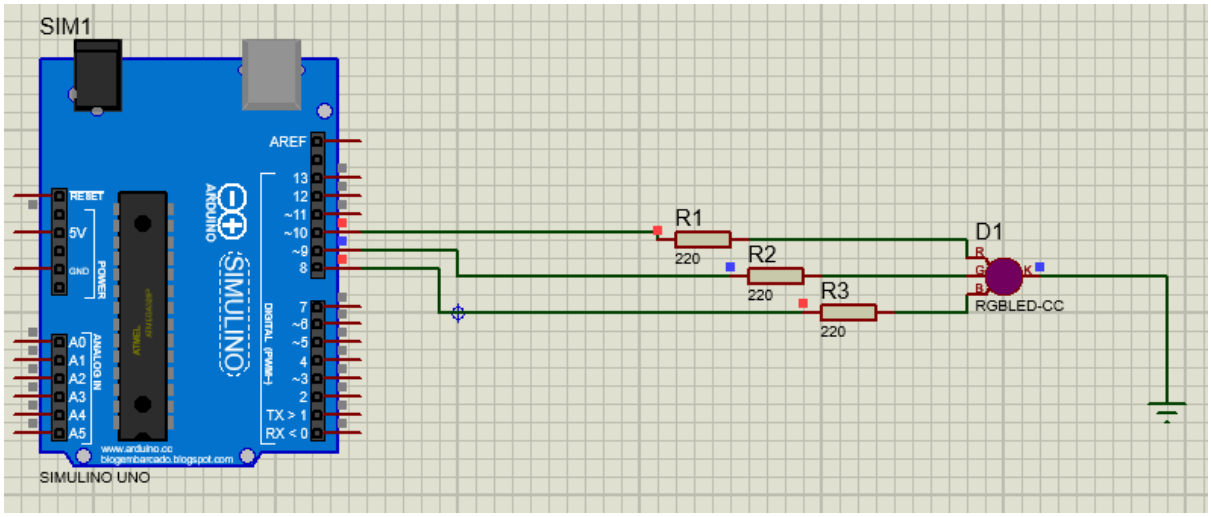
OUTPUT


SIM1
AREF
RESET
5V
GND
POWER
ANALOG IN
A0
A1
A2
A3
A4
A5
DIGITAL (PWM~)
TX > 1
RX < 0
www.arduino.cc
blogembarcado.blogspot.com
SIMULINO UNO
R1
220
R2
220
R3
220
D1
RGBLED-CC


SIM1
AREF
RESET
5V
GND
POWER
ANALOG IN
A0
A1
A2
A3
A4
A5
DIGITAL (PWM~)
TX > 1
RX < 0
www.arduino.cc
blogembarcado.blogspot.com
SIMULINO UNO
R1
220
R2
220
R3
220
D1
RGBLED-CC


SIM1
AREF
RESET
5V
GND
POWER
ANALOG IN
A0
A1
A2
A3
A4
A5
DIGITAL (PWM~)
TX > 1
RX < 0
www.arduino.cc
blogembarcado.blogspot.com
SIMULINO UNO
R1
220
R2
220
R3
220
D1
RGBLED-CC



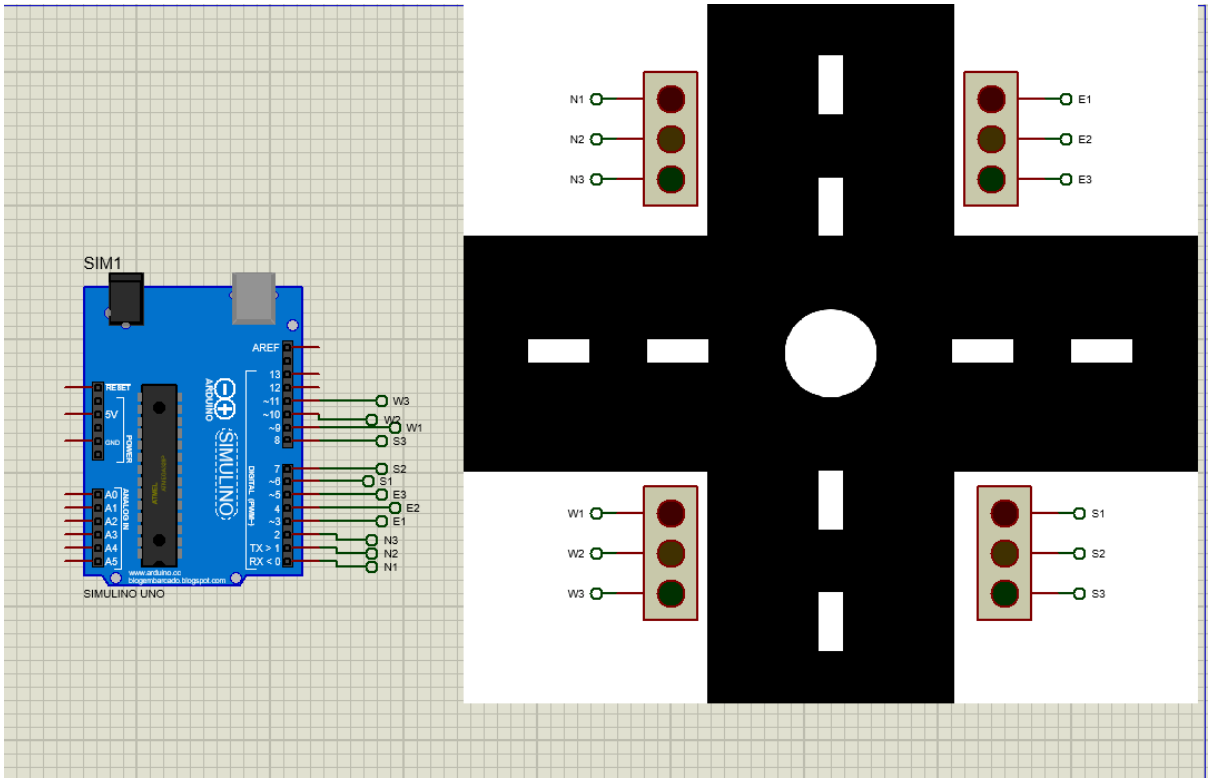


iii. Traffic Signal light Simulation

Reuirements

1. Arduino Uno
2. Image of Junction
3. Traffics Light LED -4
4. Connecting wires

Circuit Connection



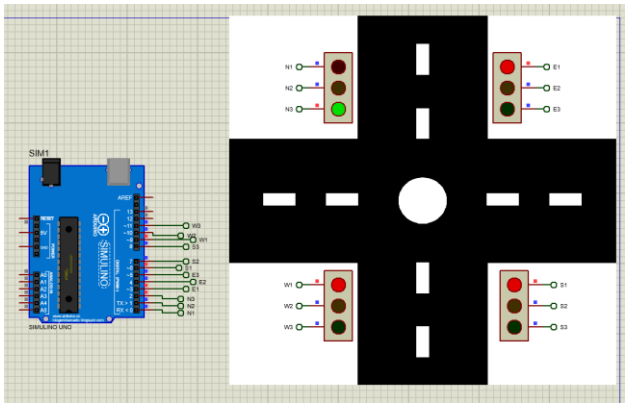
Program

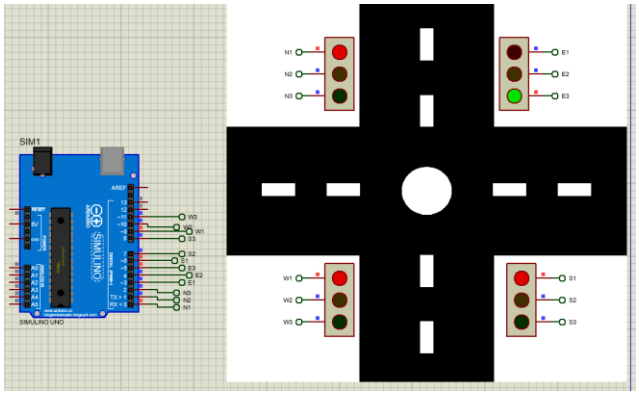
```
/*
  Traffic Light Control Project
  How its works?
  1. At the beginning, traffic light 1 will turn green for 5 seconds, and the other turn to red.
  2. After 5 seconds, traffic light 1 turns yellow for 2 seconds, then red.
  3. The same process applies to traffic lights 2, 3, 4 in seuence and will keep looping.

  The circuit:
  - Connect traffic light 1 R, Y, G, GND pins to 0, 1, 2, and GND pins of Arduino UNO
  - Connect traffic light 2 R, Y, G, GND pins to 3, 4, 5, and GND pins of Arduino UNO
  - Connect traffic light 3 R, Y, G, GND pins to 6, 7, 8, and GND pins of Arduino UNO
  - Connect traffic light 4 R, Y, G, GND pins to 9, 10, 11, and GND pins of Arduino UNO


*/
```

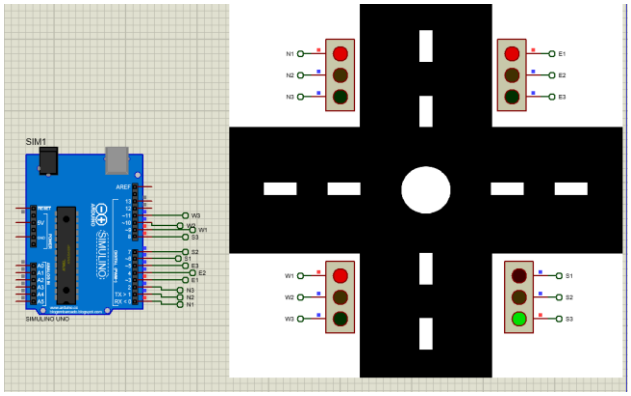
```
int timerRed = 5000;  //change your red time delay
int timerYellow = 2000;  //change your yellow time delay
int trafficPins[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11};  //an array of pin
numbers to which traffic light LED are attached
int pinCount = 12;  //the number of pins

void setup() {

  for (int thisPin = 0; thisPin < pinCount; thisPin++) {
    pinMode(trafficPins[thisPin], OUTPUT);

    switch (thisPin) {
      case 0:
      digitalWrite (trafficPins[0], HIGH);
      break;

      case 1:
      digitalWrite (trafficPins[3], HIGH);
      break;

      case 2:
      digitalWrite (trafficPins[6], HIGH);
      break;

      case 3:
      digitalWrite (trafficPins[9], HIGH);
      break;
    }

  }

}

void loop() {

  //............Traffic Light 1....................
  digitalWrite (trafficPins[0], LOW);
  digitalWrite (trafficPins[2], HIGH);

  delay (timerRed);
  digitalWrite (trafficPins[2], LOW);
  digitalWrite (trafficPins[1], HIGH);
```

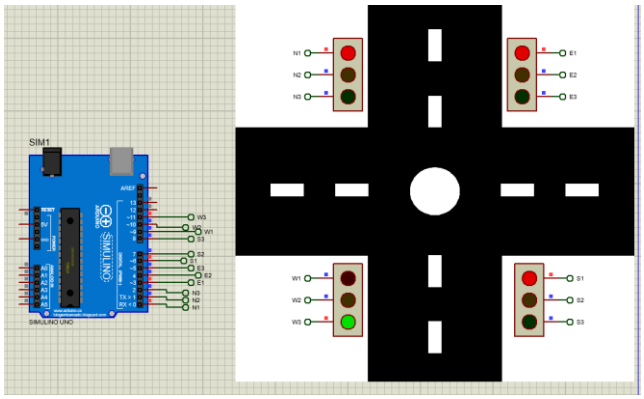
```
  delay (timerYellow);
  digitalWrite (trafficPins[1], LOW);
  digitalWrite (trafficPins[0], HIGH);

  //.............Traffic Light 2.....................
  digitalWrite (trafficPins[3], LOW);
  digitalWrite (trafficPins[5], HIGH);

  delay (timerRed);
  digitalWrite (trafficPins[5], LOW);
  digitalWrite (trafficPins[4], HIGH);
  delay (timerYellow);
  digitalWrite (trafficPins[4], LOW);
  digitalWrite (trafficPins[3], HIGH);

  //.............Traffic Light 3.....................
  digitalWrite (trafficPins[6], LOW);
  digitalWrite (trafficPins[8], HIGH);

  delay (timerRed);
  digitalWrite (trafficPins[8], LOW);
  digitalWrite (trafficPins[7], HIGH);
  delay (timerYellow);
  digitalWrite (trafficPins[7], LOW);
  digitalWrite (trafficPins[6], HIGH);

    //.............Traffic Light 4.....................
  digitalWrite (trafficPins[9], LOW);
  digitalWrite (trafficPins[11], HIGH);

  delay (timerRed);
  digitalWrite (trafficPins[11], LOW);
  digitalWrite (trafficPins[10], HIGH);
  delay (timerYellow);
  digitalWrite (trafficPins[10], LOW);
  digitalWrite (trafficPins[9], HIGH);

}
```

OUTPUT









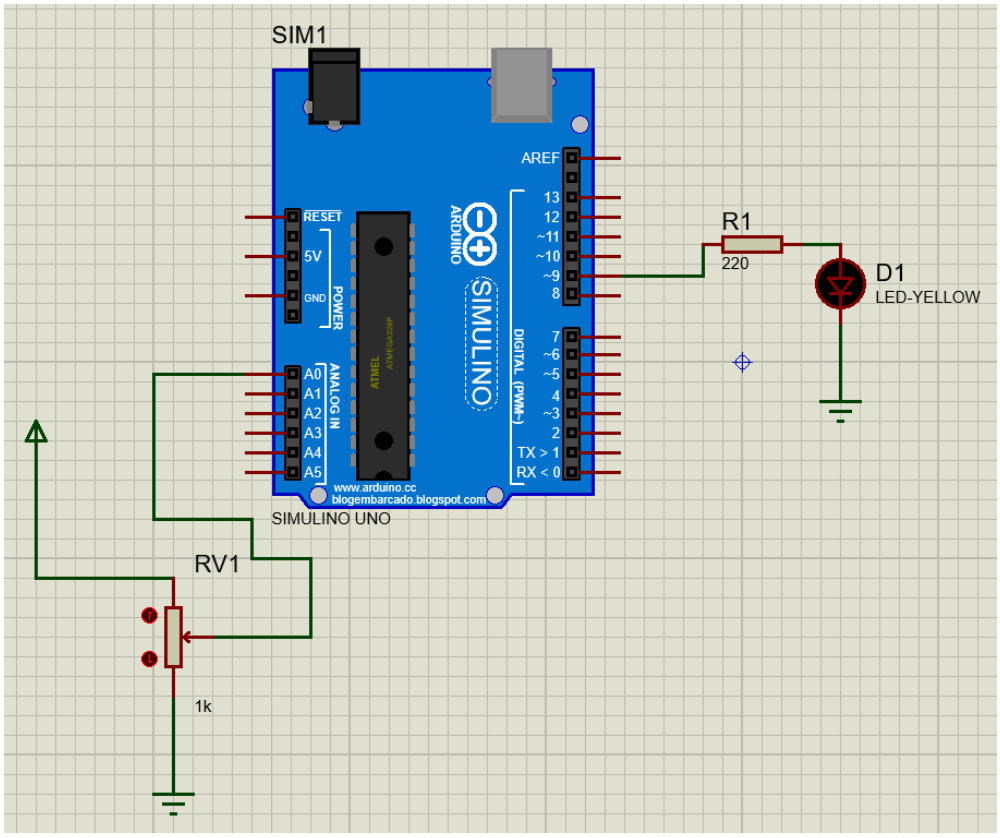
iv. Controlling the LED blink rate with the potentiometer interfacing with Arduino.

Reuirements

1. Arduino Uno -1
2. Yellow Led -1

3. Pot -1
4. Resistor-1
5. Connecting wires

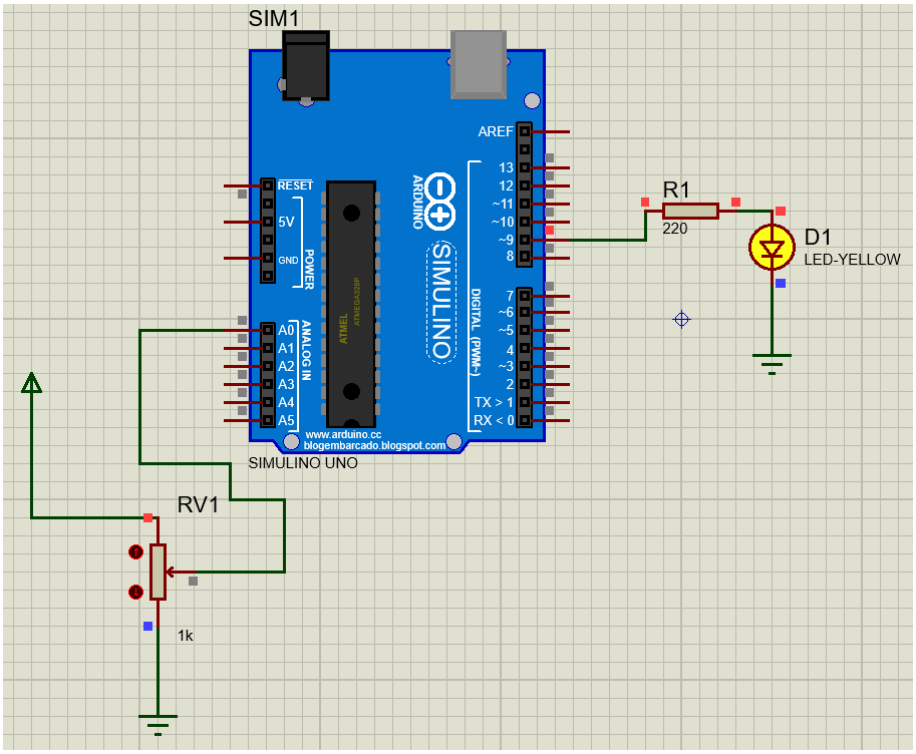
Circuit Connection

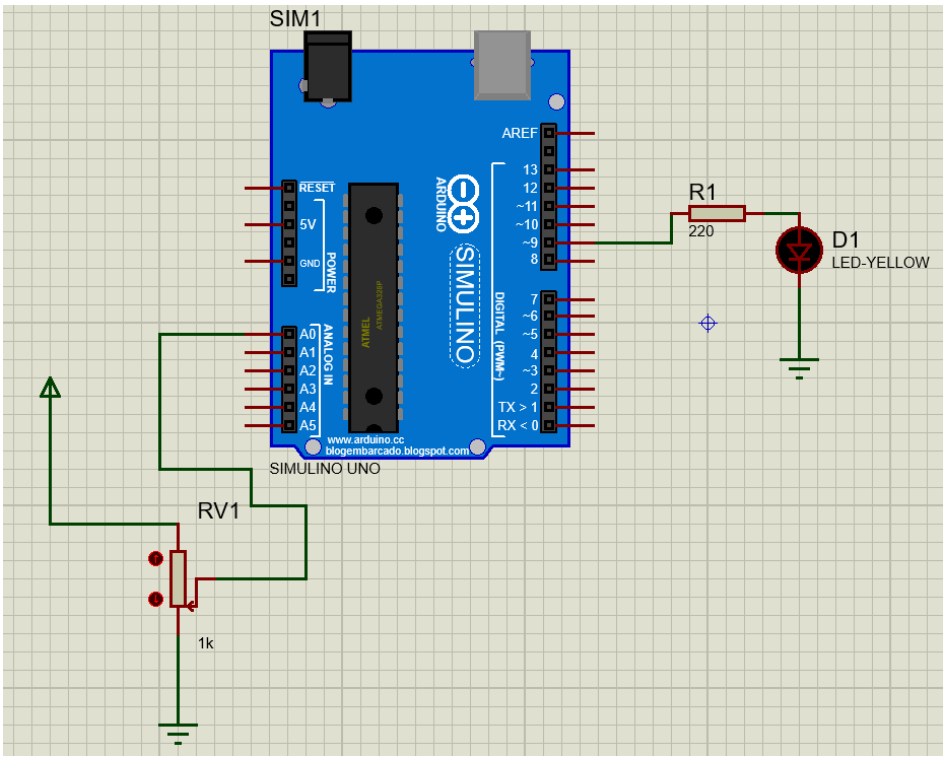


Program

```
int potpin = A0;
int ledpin=9;
int potv=0;
void setup() {
  // put your setup code here, to run once:
pinMode(ledpin, OUTPUT);
}

void loop() {
potv = analogRead(A0);
int pwm = potv / 4;

if (pwm < 10) {      // threshold
  pwm = 0;
}
analogWrite(ledpin, pwm);

}
```

OUTPUT


SIM1
AREF
RESET
5V
GND
POWER
ANALOG IN
A0
A1
A2
A3
A4
A5
ARDUINO
SIMULINO
DIGITAL (PWM~)
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX > 1
RX < 0
www.arduino.cc
blogembarcado.blogspot.com
SIMULINO UNO
R1
220
D1
LED-YELLOW
RV1
1k


SIM1
AREF
RESET
5V
GND
POWER
ANALOG IN
A0
A1
A2
A3
A4
A5
ARDUINO
SIMULINO
DIGITAL (PWM~)
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX > 1
RX < 0
www.arduino.cc
blogembarcado.blogspot.com
SIMULINO UNO
R1
220
D1
LED-YELLOW
RV1
1k

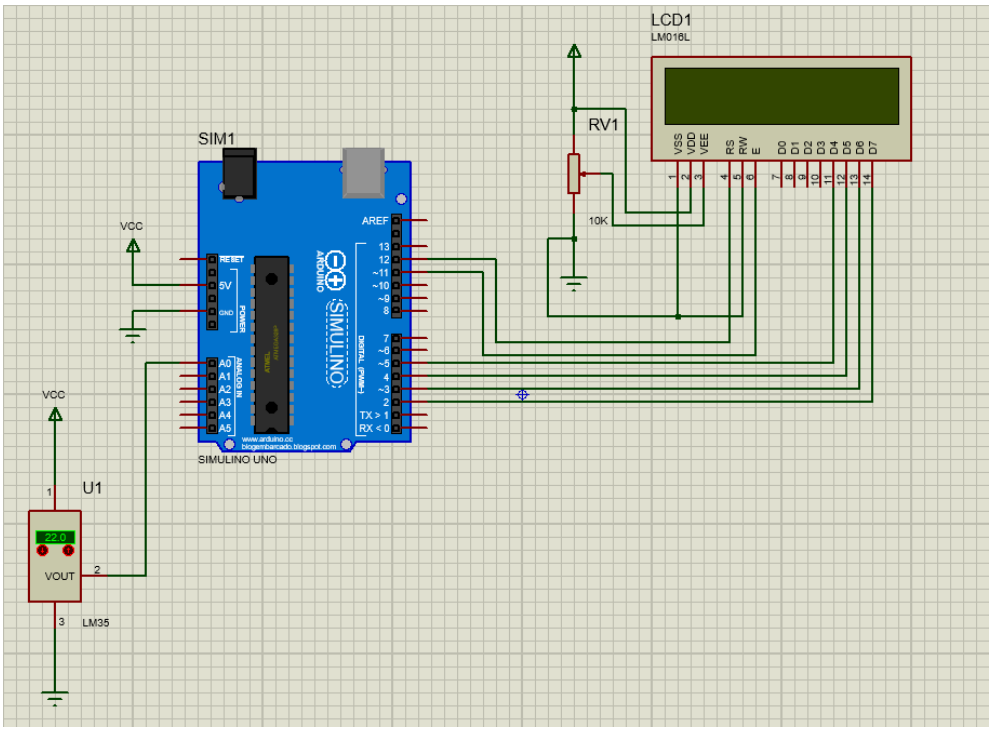
4. i. Interfacing of temperature sensor LM35 with Arduino.
   ii. Interfacing Servo Motor with the Arduino.

i. Interfacing of temperature sensor LM35 with Arduino

Reuirements

1. Arduino Uno
2. LM35 – Temperature Sensor
3. LCD – 16 x 2 Display
4. Resistor – 3005P-1-103
5. Connecting wires

Circuit Connection

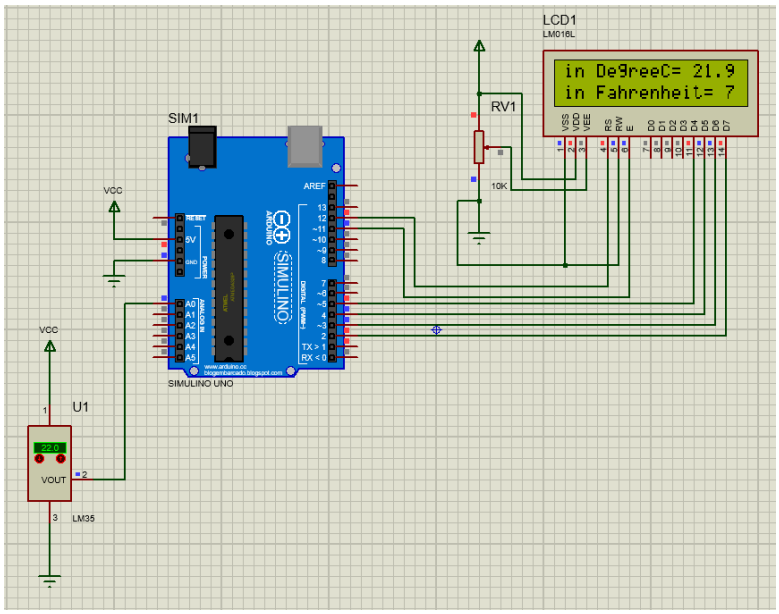


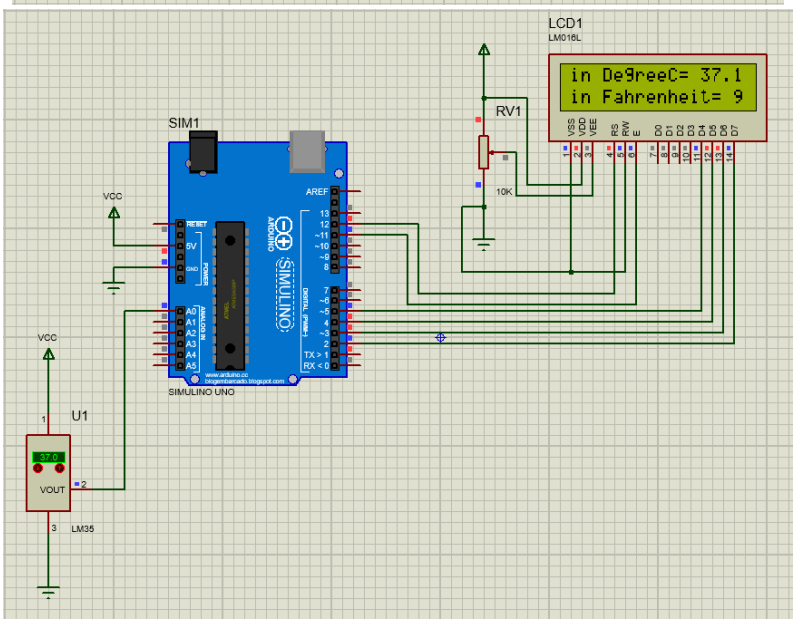
Program

```
#include<LiuidCrystal.h>
LiuidCrystal lcd(12,11,5,4,3,2);
const int sensor=A0;
float tempc;
float tempf;
float vout;
void setup()
{
pinMode(sensor,INPUT);
Serial.begin(9600);
lcd.begin(16,2);
delay(500);
}
void loop()
```

```
{
vout=analogRead(sensor);
vout=(vout*500)/1023;
tempc=vout;
tempf=(vout*1.8)+32;
lcd.setCursor(0,0);
lcd.print("in DegreeC= ");
lcd.print(tempc);
lcd.setCursor(0,1);
lcd.print("in Fahrenheit= ");
lcd.print(tempf);
delay(1000);
}
```

OUTPUT



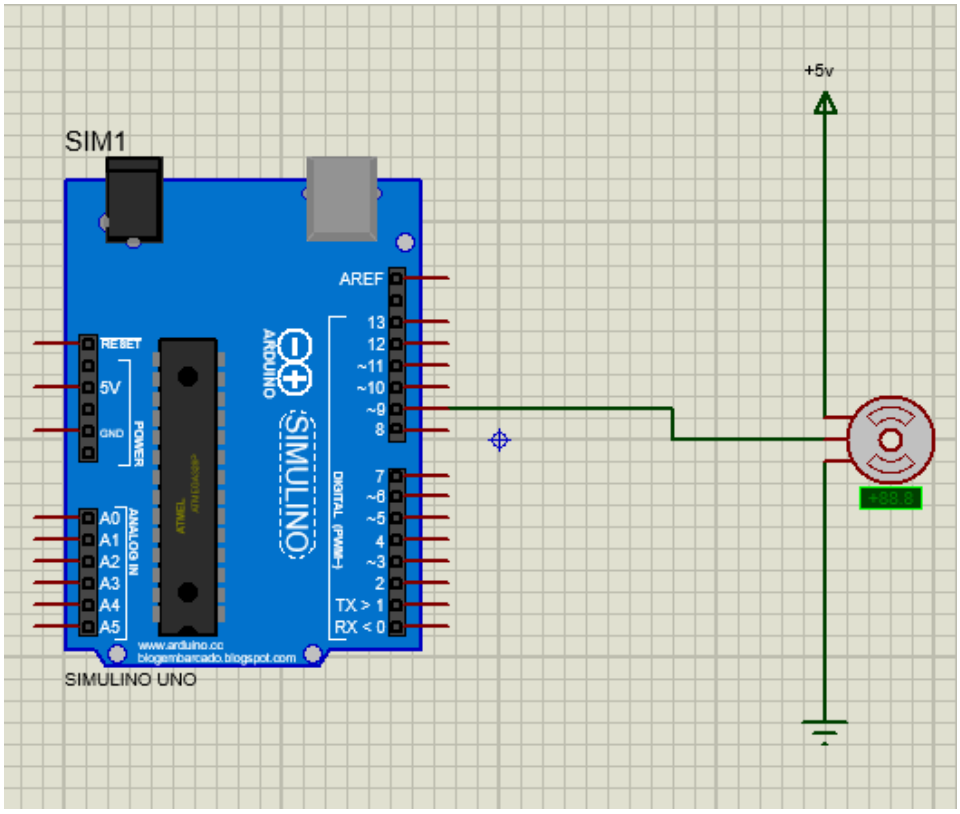


ii. Interfacing Servo Motor with the Arduino

Reuirements

1. Arduino Uno
2. Servo- Motor – PWMServo
3. Connecting wires

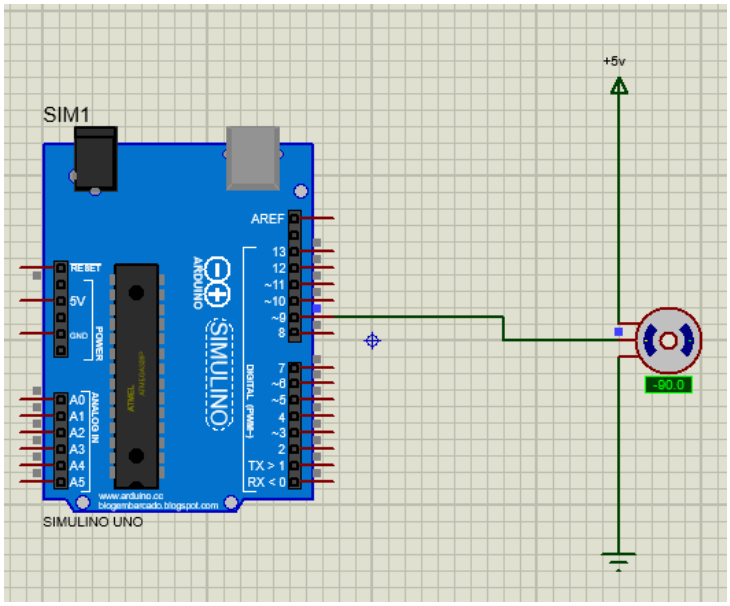
Circuit Connection

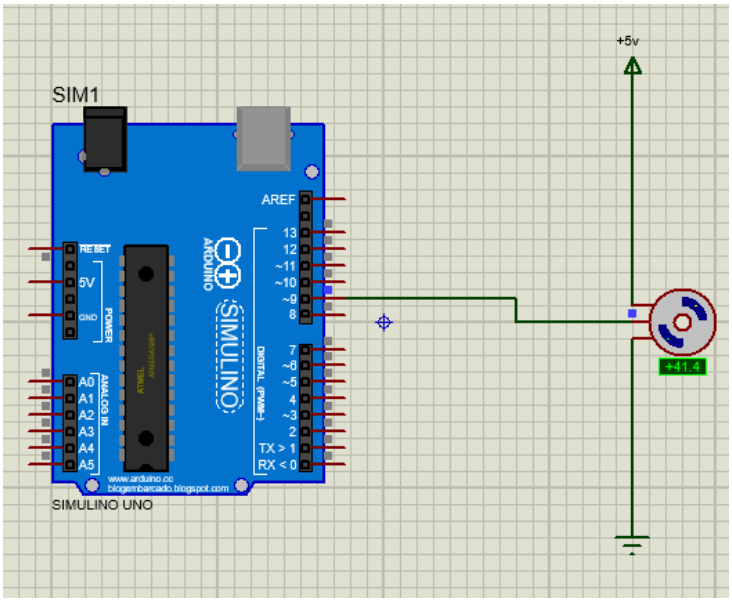


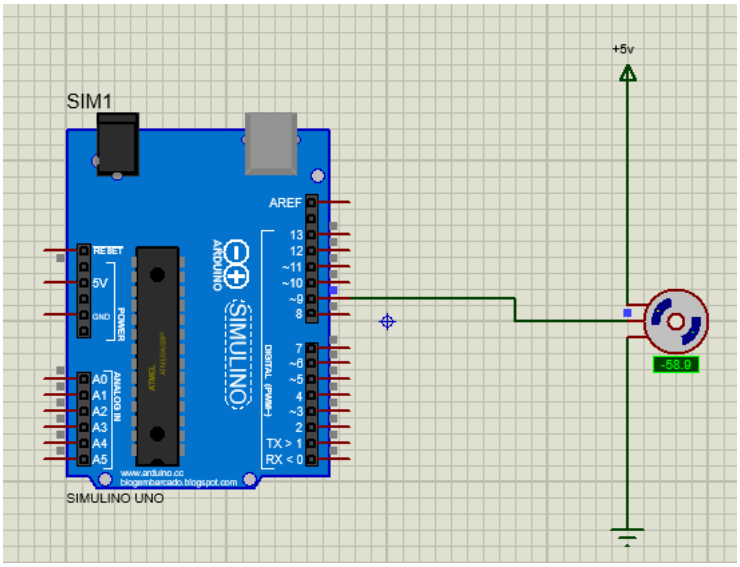
Program

```
#include <Servo.h>
Servo myservo;
int pos = 0;
void setup() {
myservo.attach(9);
}
void loop() {
for (pos = 0; pos <= 180; pos += 1) {
myservo.write(pos);
delay(15);
}
for (pos = 180; pos >= 0; pos -= 1) {
myservo.write(pos);
delay(15);
}
}
```

OUTPUT


SIM1
+5v
AREF
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX > 1
RX < 0
RESET
5V
GND
POWER
A0
A1
A2
A3
A4
A5
ANALOG IN
DIGITAL (PWM~)
SIMULINO
ARDUINO
-80.0
SIMULINO UNO


SIM1
+5v
AREF
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX > 1
RX < 0
RESET
5V
GND
POWER
A0
A1
A2
A3
A4
A5
ANALOG IN
DIGITAL (PWM~)
SIMULINO
ARDUINO
+41.4
SIMULINO UNO


SIM1
+5v
AREF
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX > 1
RX < 0
RESET
5V
GND
POWER
A0
A1
A2
A3
A4
A5
ANALOG IN
DIGITAL (PWM~)
SIMULINO
ARDUINO
-58.9
SIMULINO UNO

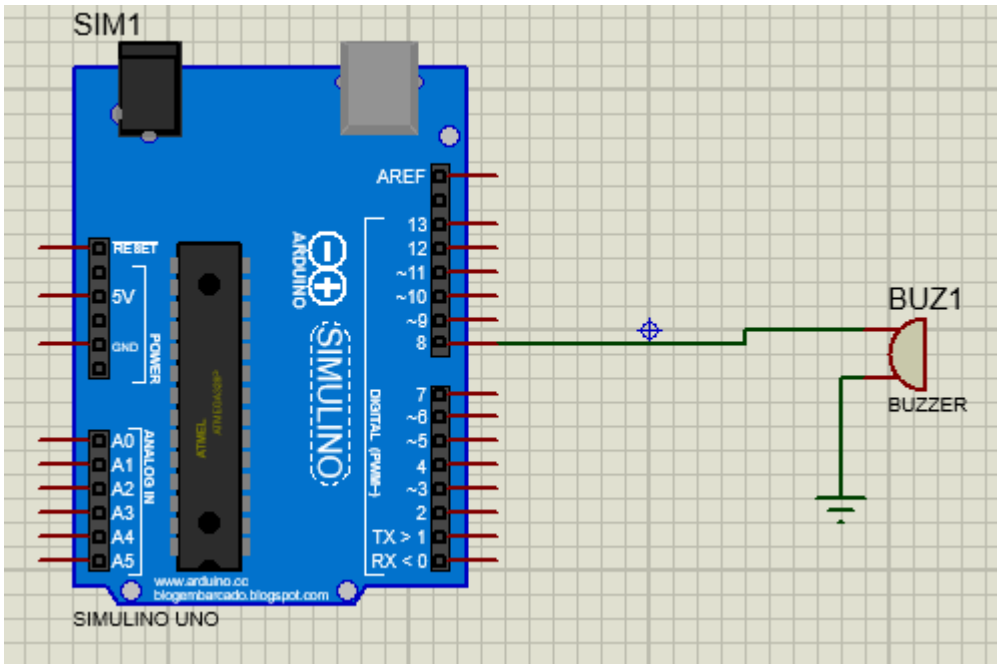
5. i. Interfacing of the Active Buzzer with Arduino.
   ii. Interfacing of the Relay with Arduino.

i. Interfacing of the Active Buzzer with Arduino.

Reuirements

1. Arduino Uno
2. Buzzer – Active one
3. Connecting wires

Circuit Connection



Note:

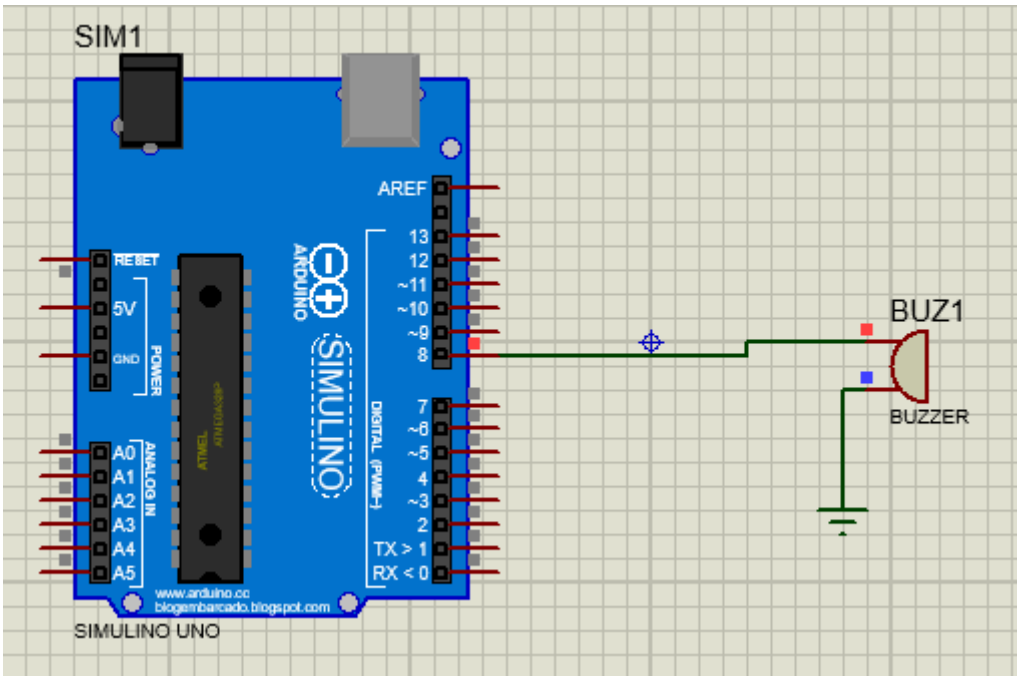
Double Click on the buzzer set the operating voltage to +5V and load Resistance to 100

Program

```
#define NOTE_A3  220
#define NOTE_B3  247
#define NOTE_C4  262
#define NOTE_G3  196
int melody[] = {
NOTE_C4, NOTE_G3, NOTE_G3, NOTE_A3, NOTE_G3, 0, NOTE_B3,
NOTE_C4
};
int noteDurations[] = {
  4, 8, 8, 4, 4, 4, 4, 4
};
void setup() {
```

```
// put your setup code here, to run once:
for (int thisNote = 0; thisNote < 8; thisNote++) {
int noteDuration = 1000 / noteDurations[thisNote];
tone(8, melody[thisNote], noteDuration);
int pauseBetweenNotes = noteDuration * 1.30;
delay(pauseBetweenNotes);
noTone(8);
}
}
void loop() {

}
```

OUTPUT

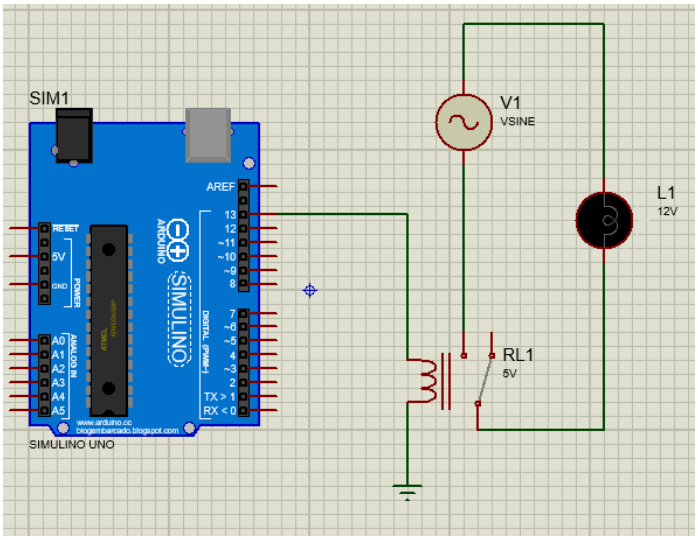


b. Interfacing of the Relay with Arduino.

Reuirements

1. Arduino Uno
2. LAMP
3. Relay- Relay [Active- Animated Relay Model]
4. VSINE- sine wave AC voltage Source
5. Connecting wires

Circuit Connection:



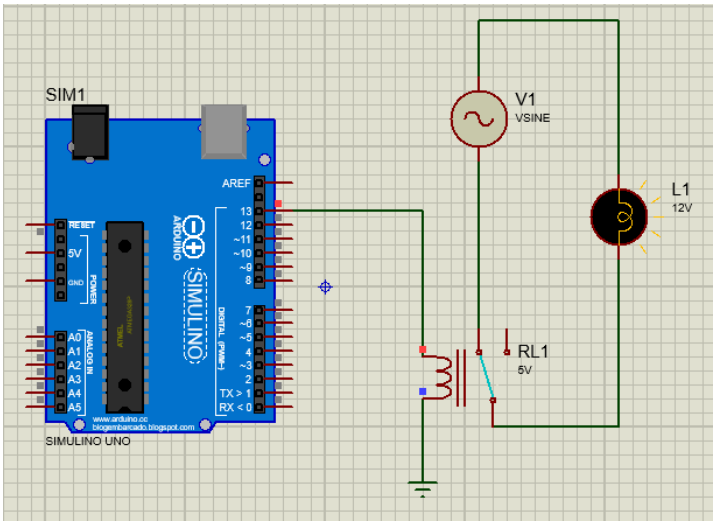
Instruction:

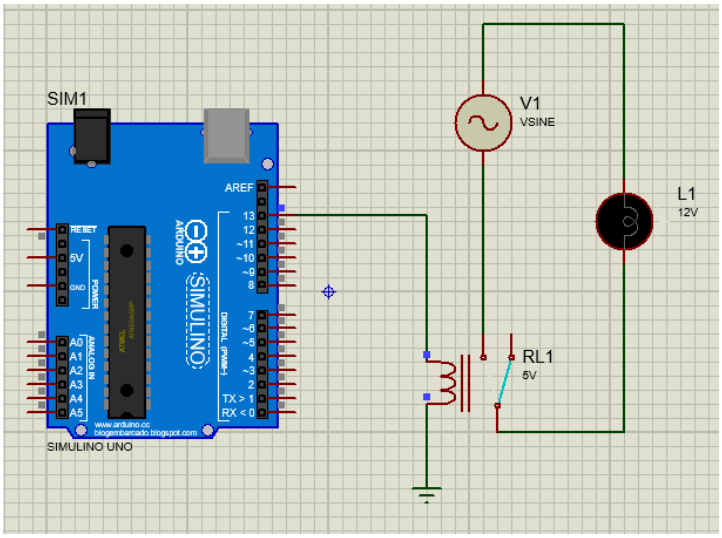
1. Double click on Relay – Change component value to 5V
2. Double click on VSINE – set the amplitude to 12V and Freuency to 5V

Program

```
void setup() {
pinMode(13, OUTPUT);
}
void loop() {
digitalWrite(13, HIGH);   // turn the LED on (HIGH is the voltage level)
delay(1000);
digitalWrite(13, LOW);    // turn the LED off by making the voltage LOW
delay(1000);
}
```

OUTPUT



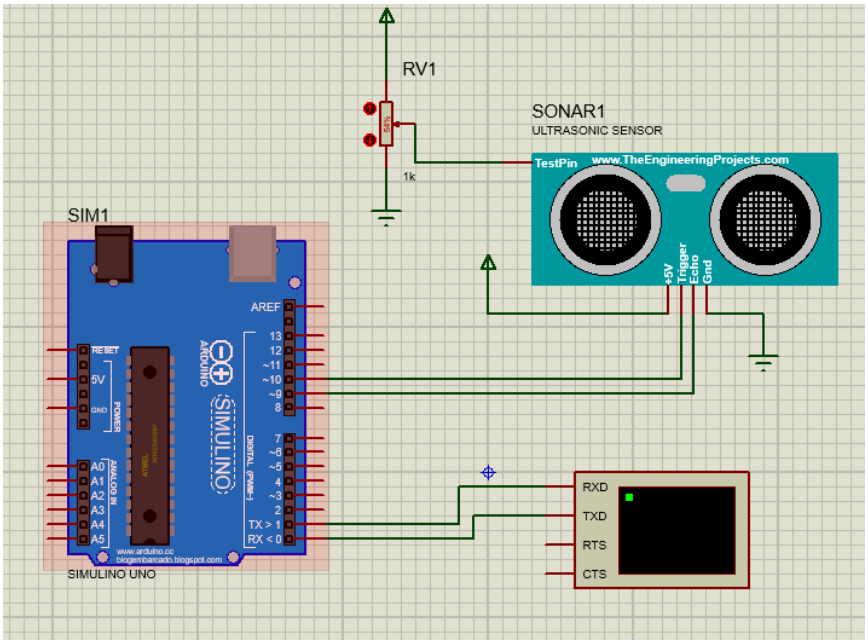

SIM1
V1
VSINE
L1
12V
RL1
5V
AREF
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX > 1
RX < 0
RESET
5V
GND
POWER
A0
A1
A2
A3
A4
A5
ANALOG IN
SIMULINO
SIMULINO UNO

6. Calculate the distance to an object with the help of an ultrasonic sensor and display it on an LCD.

Requirements

1. Arduino Uno
2. Ultrasonic Sensor
3. Virtual Display
4. Potentiometer - POT-HG
5. Connecting wires

Circuit Connection



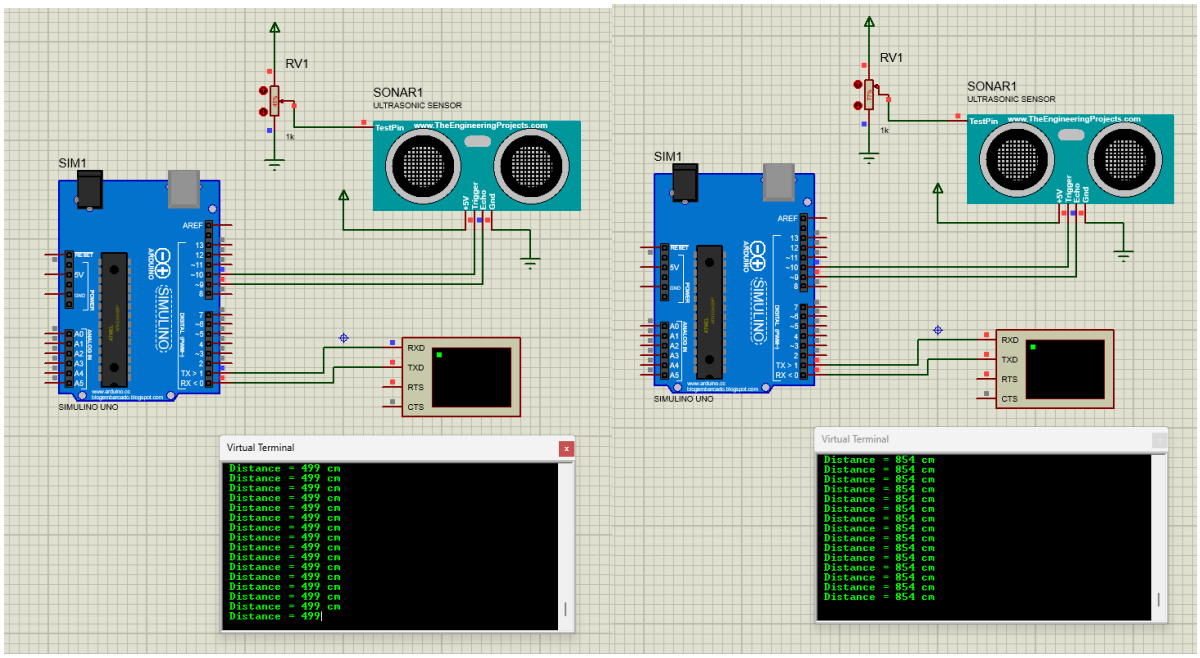
Note: Update the library file location of Ultrasonic sensor by double click select the file location were file name "UltrasonicTEP.hex" is present.

Program

```
int trig = 10; int echo = 9; long duration; int cm;
void setup() {
pinMode(trig, OUTPUT);
pinMode(echo,INPUT);
Serial.begin(9600);
}
void loop() {
digitalWrite(trig, LOW);
delayMicroseconds(10);
digitalWrite(trig, HIGH);
delayMicroseconds(10);
digitalWrite(trig, LOW);
delayMicroseconds(10);
```

```
duration = pulseIn(echo, HIGH);
cm = (duration/2) * 0.034;
Serial.print(" Distance = ");
Serial.print(cm);
Serial.println(" cm");
}
```

OUTPUT



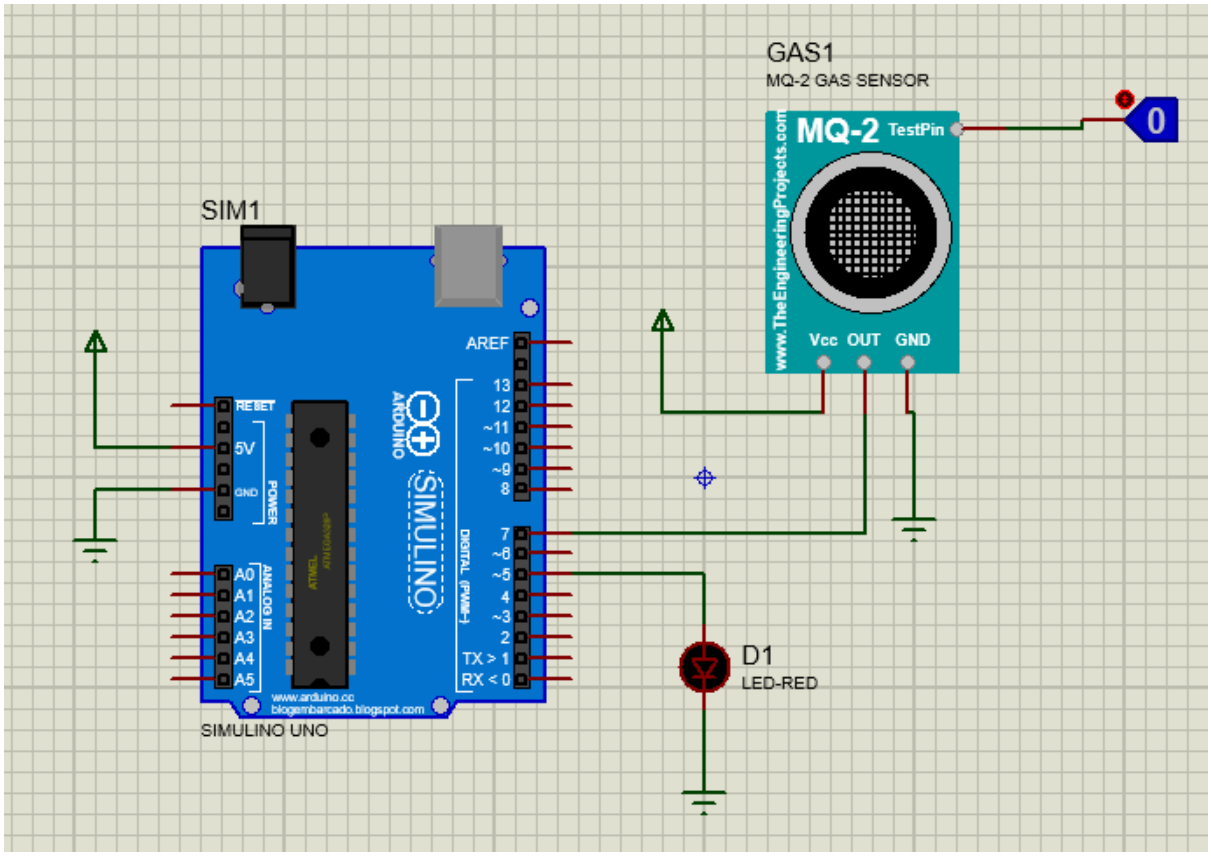
7. i. Smoke & Gas Leak Detector Using Arduino.
ii. Digital Stopwatch Using Arduino.

. i. Smoke & Gas Leak Detector Using Arduino.

Requirements

1. Arduino Uno
2. Gas Sensor- M-2 Gas Sensor
3. led – any colour
4. Logic toggle
5. Connecting wires

Circuit Connection



Instruction:

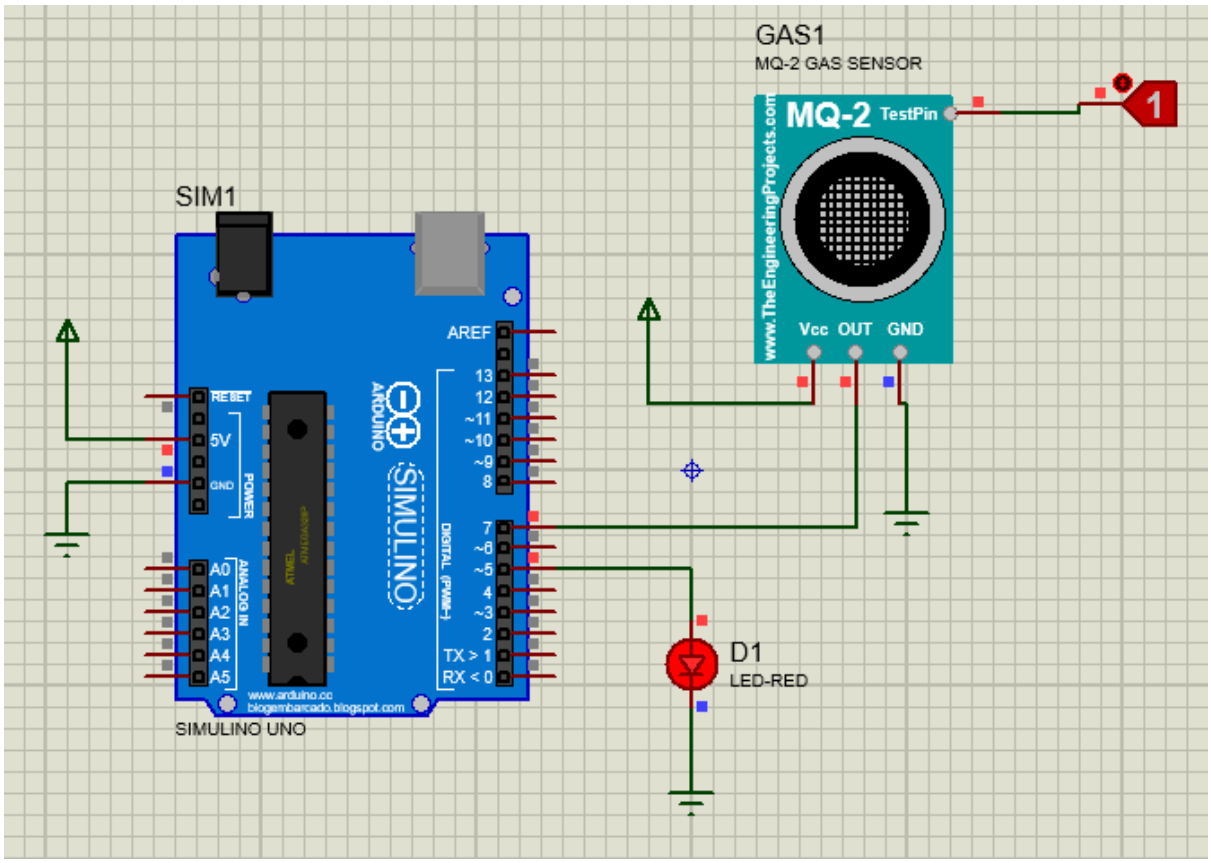
Update the library file location of M 2 Gas sensor by double click select the file location were file name "GasSensorTEP.hex" is present.

Program:

```
const int Sesnsor_Pin = 7; // Sensor Pin
const int LED_Pin = 5; // LED Pin
void setup()
{
  pinMode(Sesnsor_Pin,INPUT);  // Sensor Pin as a INPUT
  pinMode(LED_Pin,OUTPUT);  // LED Pin as a OUTPUT
```

```
}
void loop()
{
  if(digitalRead(Sesnsor_Pin) == HIGH)  // If Sensor Detect the Smoke/Gas
  {
    digitalWrite(LED_Pin,HIGH);  // Turn ON the LED
  }
  else
  {
    digitalWrite(LED_Pin,LOW);  // Turn OFF the LED
  }
}
```

OUTPUT

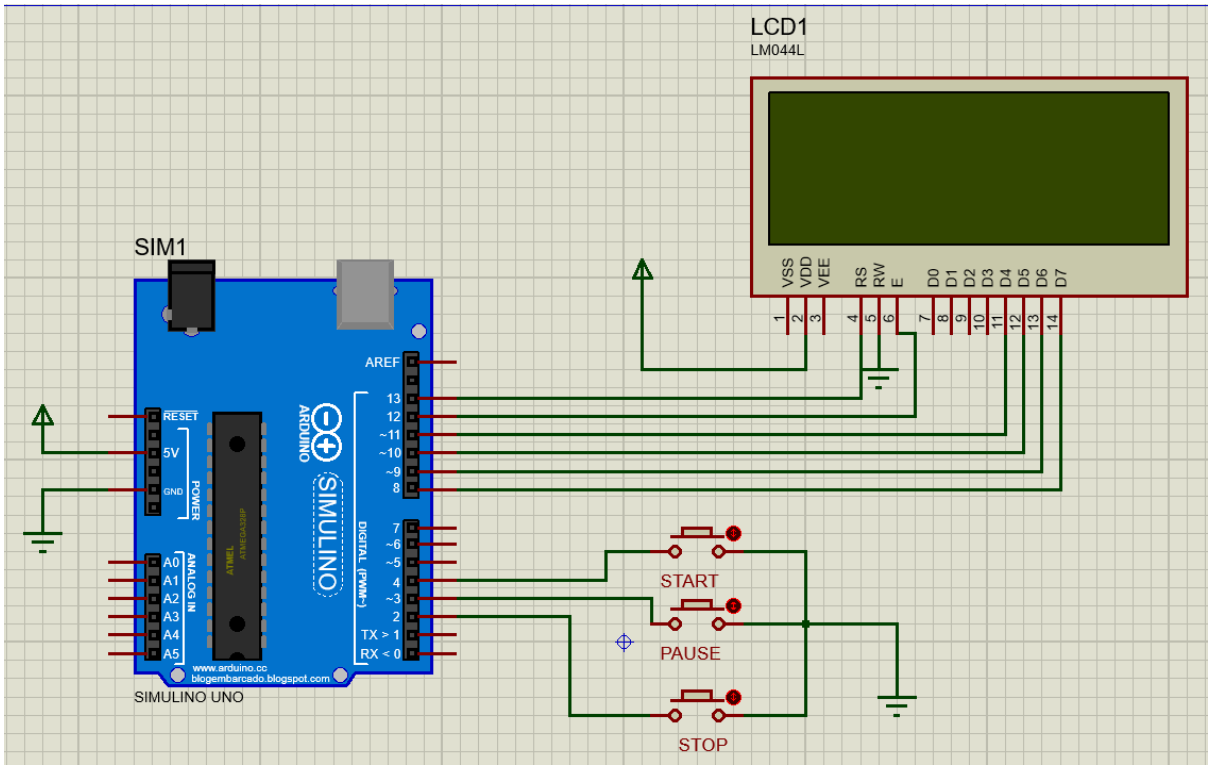


ii. Digital Stopwatch Using Arduino.

Requirements

1. Arduino Uno
2. LCD – LM044L
3. Button
4. Connecting wires

Circuit Connection



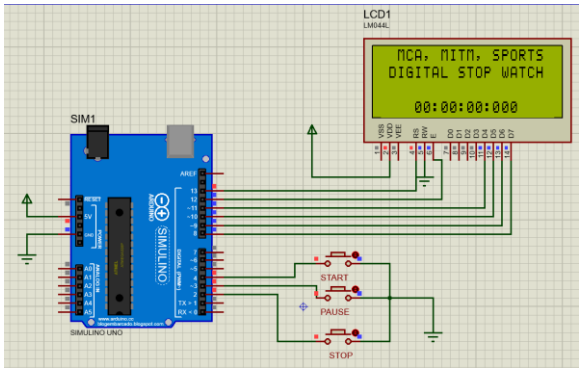
Program:

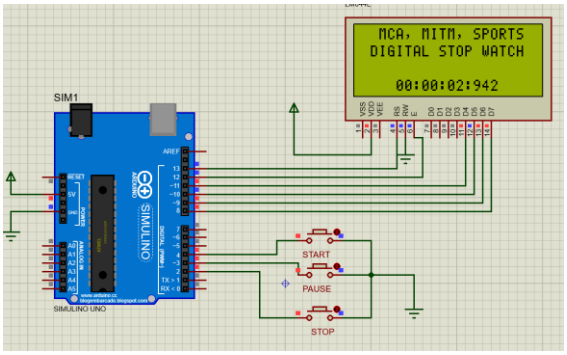
```
#include <LiuidCrystal.h>

//LiuidCrystal lcd(rs, en, d4, d5, d6, d7);
LiuidCrystal lcd(13, 12, 11, 10,  9,  8);

const int bt_start = 4;  //Start Button Pin
const int bt_pause = 3;  //Pause Button Pin
const int bt_reset = 2;  //Reset Button Pin

int hrs = 0;  //Hours
int mnt = 0;  //Minutes
int sec = 0;  //Seconds
int msec = 0;  //Milli Seconds

bool Timer = false;

void setup()
{
  pinMode(bt_start, INPUT_PULLUP);
  pinMode(bt_pause,  INPUT_PULLUP);
  pinMode(bt_reset, INPUT_PULLUP);
```

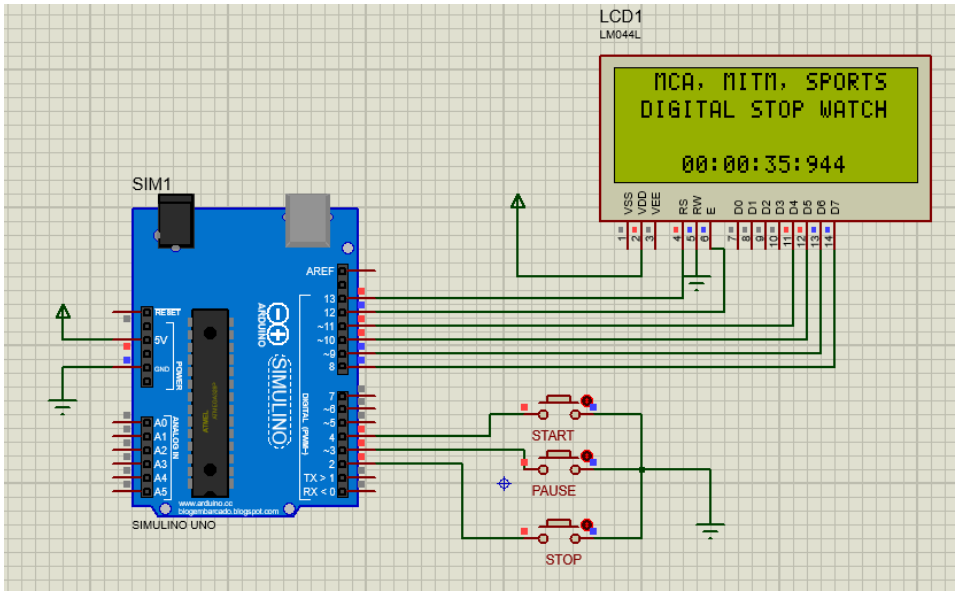
```
  lcd.begin(20, 4); // set up the LCD's number of columns and rows:
  lcd.setCursor(0,0); // set the cursor position:
  lcd.print("  MCA, MITM, SPORTS");
  lcd.setCursor(0,1);
  lcd.print(" DIGITAL STOP WATCH              ");

  noInterrupts();   // disable all interrupts
  TCCR1A = 0;      // set entire TCCR1A register to 0  //set timer1 interrupt
at 1kHz  // 1 ms
  TCCR1B = 0;      // same for TCCR1B
  TCNT1  = 0;       // set timer count for 1khz increments
  OCR1A = 1999;     // = (16*10^6) / (1000*8) - 1
  //had to use 16 bit timer1 for this bc 1999>255, but could switch to
timers 0 or 2 with larger prescaler
  // turn on CTC mode
  TCCR1B |= (1 << WGM12); // Set CS11 bit for 8 prescaler
  TCCR1B |= (1 << CS11);  // enable timer compare interrupt
  TIMSK1 |= (1 << OCIE1A);
  interrupts();   // enable
}

void loop()
{
  if (digitalRead (bt_start) == 0)
  {
    Timer = true; // Start stopwatch
    delay(100);
  }

  if (digitalRead (bt_pause) == 0)
  {
    Timer = false; // Stop stopwatch
    delay(100);
  }

  if (digitalRead (bt_reset) == 0)
  {
    // Reset stopwatch
    hrs = 0;
    mnt = 0;
    sec = 0;
    msec = 0;
```

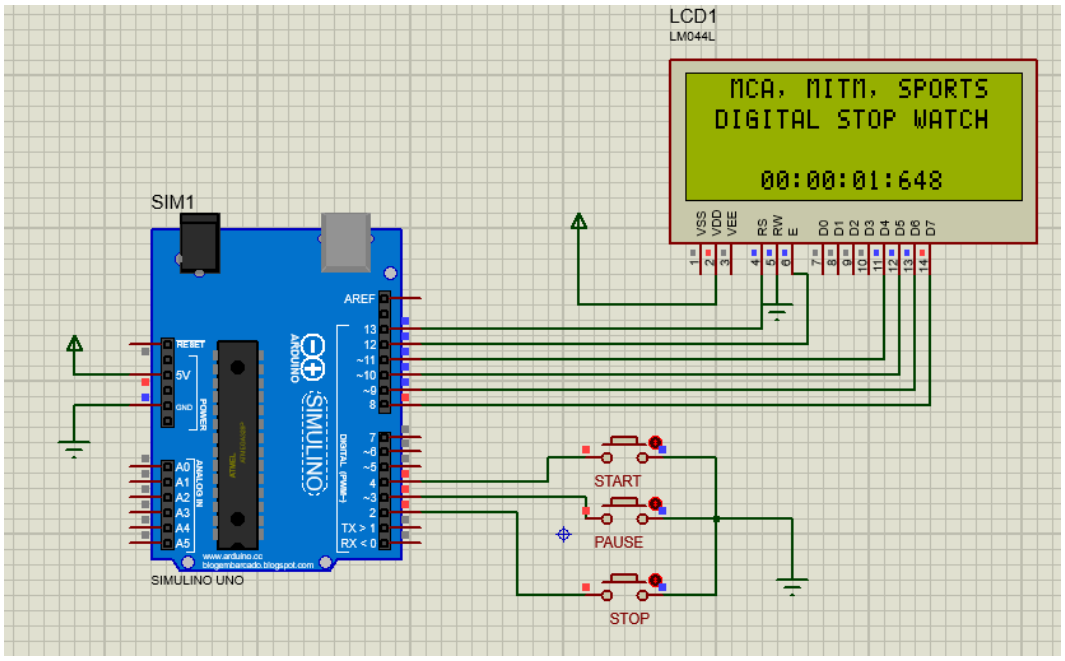
```
    delay(100);
  }

  lcd.setCursor (4, 3);
  lcd.print((hrs / 10) % 10);
  lcd.print(hrs % 10);
  lcd.print(":");
  lcd.print((mnt / 10) % 10);
  lcd.print(mnt % 10);
  lcd.print(":");
  lcd.print((sec / 10) % 10);
  lcd.print(sec % 10);
  lcd.print(":");
  lcd.print((msec / 100) % 10);
  lcd.print((msec / 10) % 10);
  lcd.print(msec % 10);
}

ISR(TIMER1_COMPA_vect)
{
  if (Timer == true)
  {
    msec = msec + 1;

    if (msec > 999)
    {
      msec = 0;
      sec = sec + 1;

      if (sec > 59)
      {
        sec = 0;
        mnt = mnt + 1;
      }
      if (mnt > 59)
      {
        mnt = 0;
        hrs = hrs + 1;
      }
    }
  }
}
```

OUTPUT


LCD1
LM044L
MCA, MITM, SPORTS
DIGITAL STOP WATCH
00:00:00:000
SIM1
START
PAUSE
STOP


MCA, MITM, SPORTS
DIGITAL STOP WATCH
00:00:02:942
SIM1
START
PAUSE
STOP


LCD1
LM044L
MCA, MITM, SPORTS
DIGITAL STOP WATCH
00:00:35:944
SIM1
AREF
SIMULINO UNO
START
PAUSE
STOP

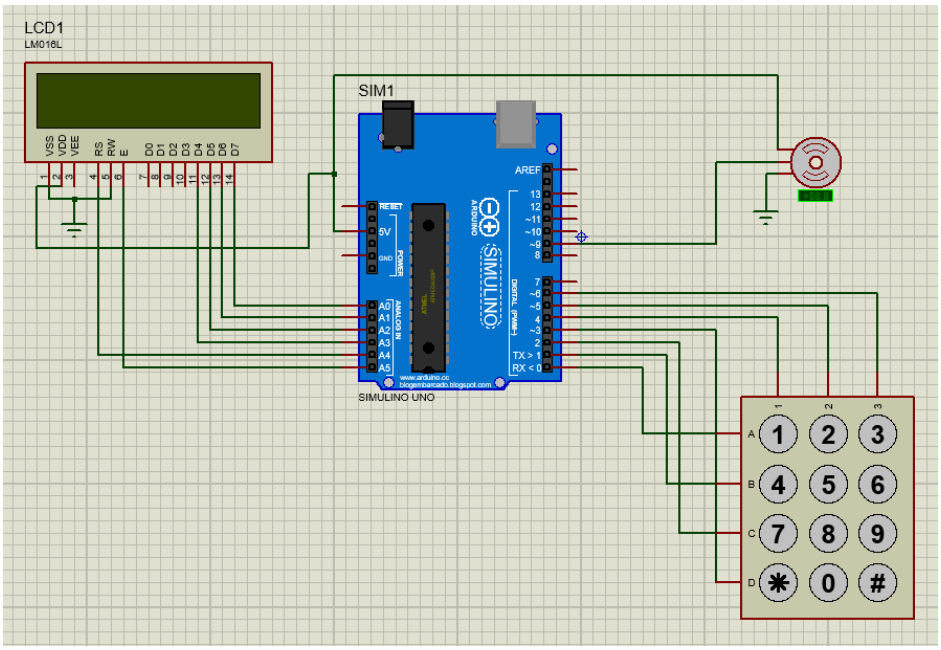

LCD1
LM044L
MCA, MITM, SPORTS
DIGITAL STOP WATCH
00:00:01:648
SIM1
AREF
SIMULINO UNO
START
PAUSE
STOP

8. Password Based Door Lock System Using Arduino.

Requirements

1. Arduino Uno
2. LCD – LM016L
3. Motor-PWMSERVO
4. Keypad-PHONE
5. Connecting wires

Circuit Connection



Instruction: In Arduino IDE, Include the Keyboard Lib to IDE

Program

```
#include <LiuidCrystal.h>
#include <Servo.h>
#include <Keypad.h>
Servo myservo;
int pos=0; // position of servo motor
LiuidCrystal lcd(A4, A5, A3, A2, A1, A0);
const byte rows=4;
const byte cols=3;

char key[rows][cols]={
{'1','2','3'},
{'4','5','6'},
{'7','8','9'},
{'*','0','#'}
};
byte rowPins[rows]={0,1,2,3};
byte colPins[cols]={4,5,6};
```

```
Keypad keypad= Keypad(makeKeymap(key),rowPins,colPins,rows,cols);
char* password="0123";
int currentposition=0;

void setup()
{
 displayscreen();
//Serial.begin(9600);
myservo.attach(9); //Servo motor connection
 lcd.begin(16,2);
 }

void loop()
{
if( currentposition==0)
{
displayscreen();

}
int l ;
char code=keypad.getKey();
if(code!=NO_KEY)
{
lcd.clear();
lcd.setCursor(0,0);
lcd.print("PASSWORD:");
lcd.setCursor(7,1);
lcd.print(" ");
lcd.setCursor(7,1);
for(l=0;l<=currentposition;++l)
{
lcd.print("*");
//keypress();
}
if (code==password[currentposition])
{
++currentposition;
if(currentposition==4)
{
unlockdoor();
currentposition=0;
}
```

```
}
else
{
incorrect();
currentposition=0;

}
}
}
//------------------ Function 1- OPEN THE DOOR--------------//
void unlockdoor()
{
delay(900);

lcd.setCursor(0,0);
lcd.println(" ");
lcd.setCursor(1,0);
lcd.print("Access Granted");
lcd.setCursor(4,1);
lcd.println("WELCOME!!");
lcd.setCursor(15,1);
lcd.println(" ");
lcd.setCursor(16,1);
lcd.println(" ");
lcd.setCursor(14,1);
lcd.println(" ");
lcd.setCursor(13,1);
lcd.println(" ");

for(pos = 180; pos>=0; pos-=5) // open the door
{
myservo.write(pos);
delay(5);
}
delay(2000);
delay(1000);
counterbeep();
delay(1000);
for(pos = 0; pos <= 180; pos +=5) // close the door
{ // in steps of 1 degree
myservo.write(pos);
delay(15);
```

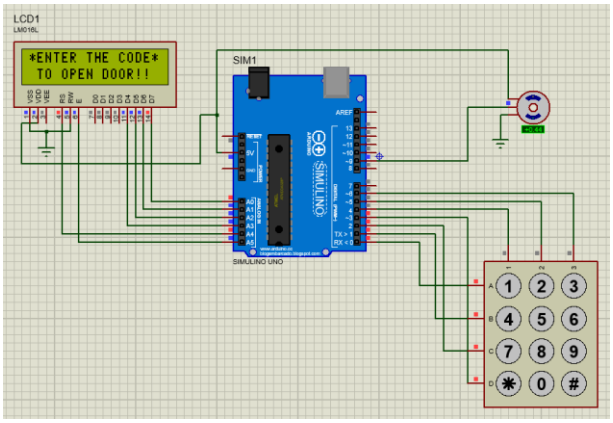
```
 currentposition=0;

lcd.clear();
displayscreen();

}
}
//--------------------Function 2- Wrong code--------------//

void incorrect()
{
delay(500);
lcd.clear();
lcd.setCursor(1,0);
lcd.print("CODE");
lcd.setCursor(6,0);
lcd.print("INCORRECT");
lcd.setCursor(15,1);
lcd.println(" ");
lcd.setCursor(4,1);
lcd.println("GET AWAY!!!");

lcd.setCursor(13,1);
lcd.println(" ");
Serial.println("CODE INCORRECT YOU ARE UNAUTHORIZED");
delay(3000);
lcd.clear();
displayscreen();
}
//-------Function 3 - CLEAR THE SCREEN--------------------/
void clearscreen()
{
lcd.setCursor(0,0);
lcd.println(" ");
lcd.setCursor(0,1);
lcd.println(" ");
lcd.setCursor(0,2);
lcd.println(" ");
lcd.setCursor(0,3);
lcd.println(" ");
}
//------------Function 4 - DISPLAY FUNCTION--------------------//
```

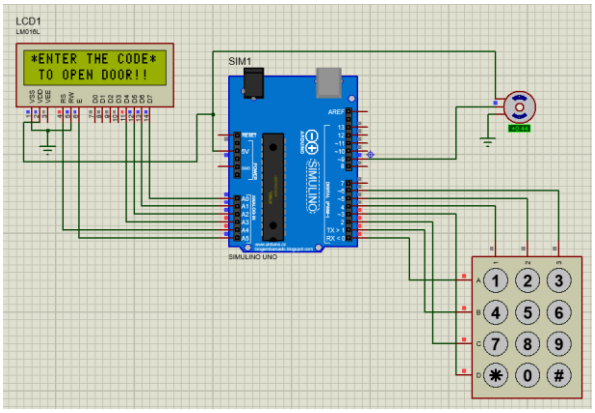
```
void displayscreen()
{

lcd.setCursor(0,0);
lcd.println("*ENTER THE CODE*");
lcd.setCursor(1 ,1);

lcd.println("TO OPEN DOOR!!");
}

//--------------Function 5 - Count down------------------//
void counterbeep()
{
delay(1200);


lcd.clear();

lcd.setCursor(2,15);
lcd.println(" ");
lcd.setCursor(2,14);
lcd.println(" ");
lcd.setCursor(2,0);
delay(200);
lcd.println("GET IN WITHIN:::");

lcd.setCursor(4,1);
lcd.print("5");
delay(200);
lcd.clear();
lcd.setCursor(2,0);
lcd.println("GET IN WITHIN:");
delay(1000);
lcd.setCursor(2,0);
lcd.println("GET IN WITHIN:");
lcd.setCursor(4,1); //2
lcd.print("4");
delay(100);
lcd.clear();
lcd.setCursor(2,0);
lcd.println("GET IN WITHIN:");
delay(1000);
```

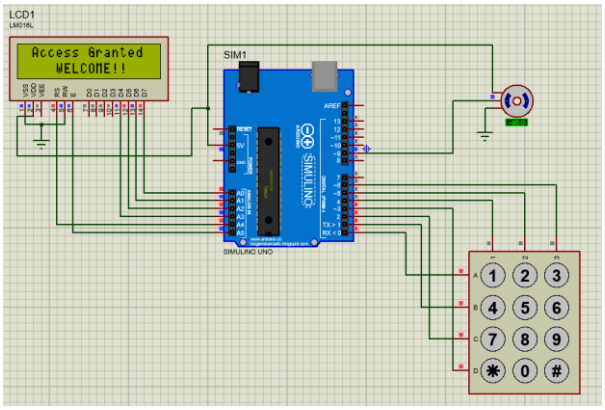
```
lcd.setCursor(2,0);
lcd.println("GET IN WITHIN:");
lcd.setCursor(4,1);
lcd.print("3");
delay(100);
lcd.clear();
lcd.setCursor(2,0);
lcd.println("GET IN WITHIN:");
delay(1000);

lcd.setCursor(2,0);
lcd.println("GET IN WITHIN:");
lcd.setCursor(4,1);
lcd.print("2");
delay(100);
lcd.clear();
lcd.setCursor(2,0);
lcd.println("GET IN WITHIN:");
delay(1000);

lcd.setCursor(4,1);
lcd.print("1");
delay(100);
lcd.clear();
lcd.setCursor(2,0);
lcd.println("GET IN WITHIN::");

delay(1000);
delay(40);
lcd.clear();
lcd.setCursor(2,0);
lcd.print("RE-LOCKING");
delay(500);
lcd.setCursor(12,0);
lcd.print(".");
delay(500);
lcd.setCursor(13,0);
lcd.print(".");
delay(500);
lcd.setCursor(14,0);
lcd.print(".");
```

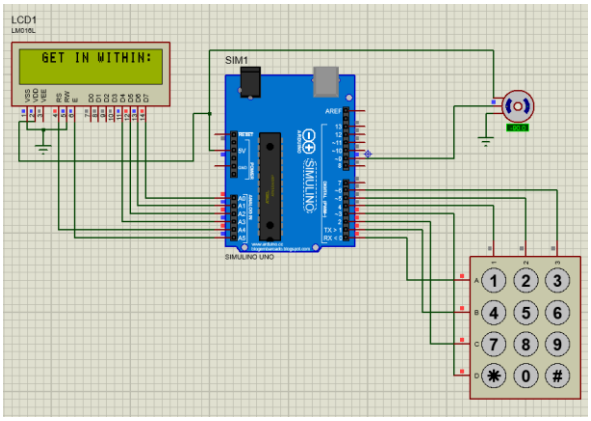
```
delay(400);
lcd.clear();
lcd.setCursor(4,0);
lcd.print("LOCKED!");
delay(440);
}
```

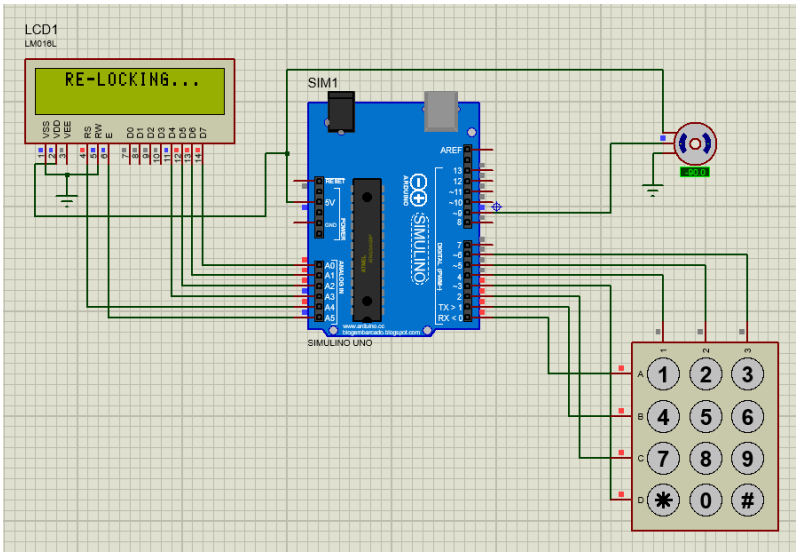
OUTPUT









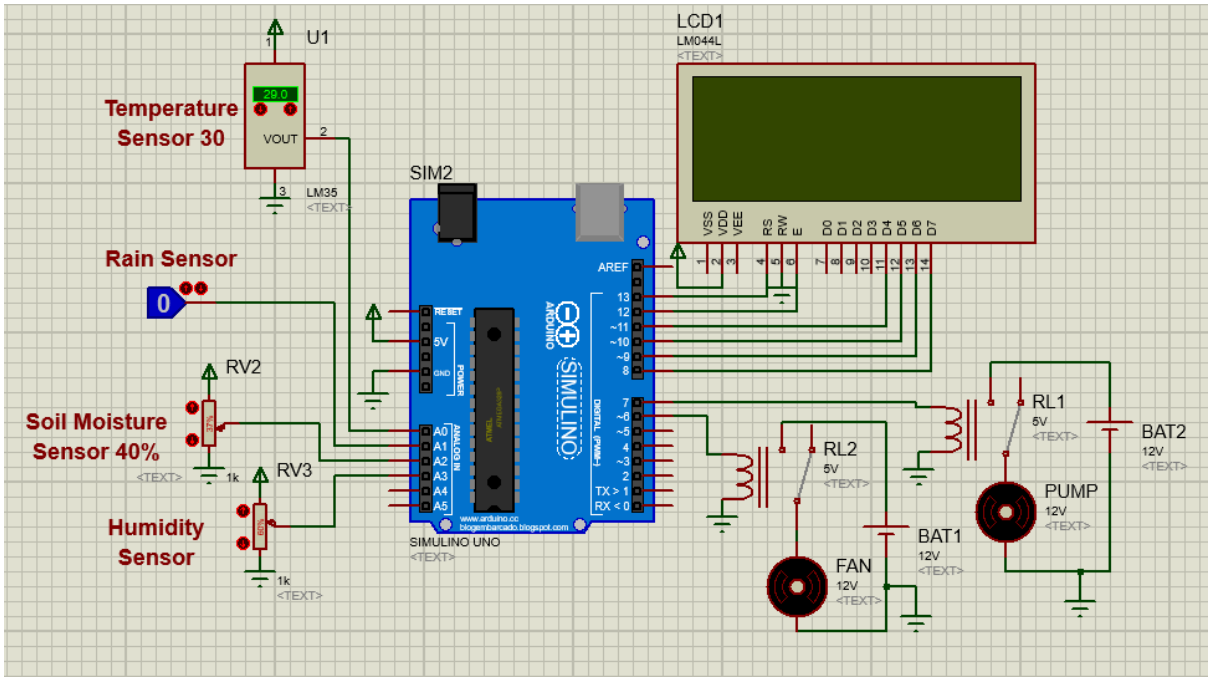


9. IoT Based Smart Farming System

Requirements

1. Arduino Uno
2. LCD – LM044L
3. CELL
4. LM35
5. LOGICSTATE
6. MOTOR
7. RELAY
8. POT-HG
8. Connecting wires

Circuit Connection



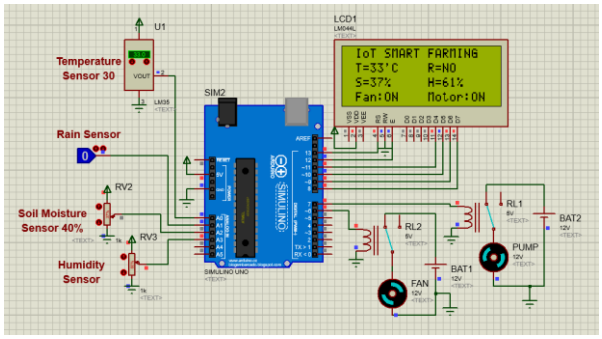
Program

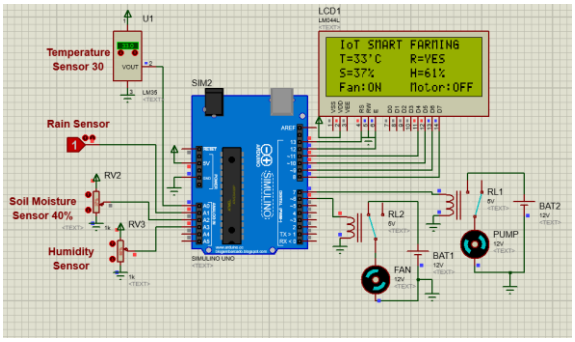
```
#include <LiuidCrystal.h>
//LiuidCrystal lcd(rs, en, d4, d5, d6, d7);
  LiuidCrystal lcd(13, 12, 11, 10,  9,  8);
const int Fan_Pin = 6;
const int Pump_Pin = 7;
void setup()
{
  lcd.begin(20, 4); // set up the LCD's number of columns and rows:
  lcd.setCursor(0,0); // set the cursor position:
  lcd.print(" IoT SMART FARMING    ");

  pinMode(Fan_Pin,OUTPUT);
  pinMode(Pump_Pin,OUTPUT);
  pinMode(A1,INPUT);
```

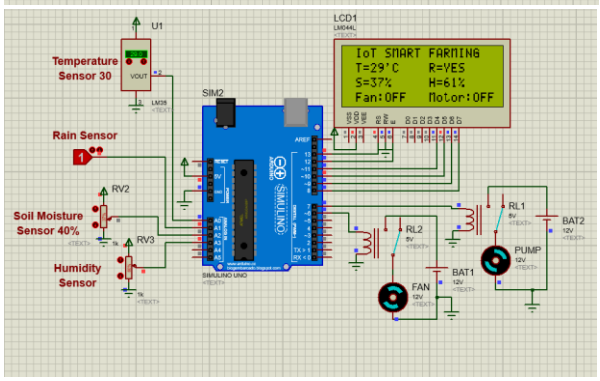
```
}
void loop()
{
  //Temperature Sensing
  int S1=analogRead(A0);  // Read Temperature
  float mV=(S1/1023.0)*5000; // Storing value in Degree Celsius
  int Temp = mV/10;
  lcd.setCursor(0,1);
  lcd.print(" T=");
  lcd.print(Temp);
  lcd.print("'C  ");
  //Rain Sensing
  int R=digitalRead(A1);  // Read Rain Sensor
  if(R==0)
  {
    lcd.setCursor(10,1);
    lcd.print(" R=NO ");
  }
  if(R==1)
  {
    lcd.setCursor(10,1);
    lcd.print(" R=YES ");
  }
  //Soil Moisture
  int S3=analogRead(A2);  // Read Soil Moisture
  int SM=S3/10;
  lcd.setCursor(0,2);
  lcd.print(" S=");
  lcd.print(SM);
  lcd.print("%  ");

  //Air Humidity
  int S4=analogRead(A3);  // Read Air Humidity
  int H=S4/10;
  lcd.setCursor(10,2);
  lcd.print(" H=");
  lcd.print(H);
  lcd.print("%  ");

  if(Temp>30)
  {
    digitalWrite(Fan_Pin,HIGH);
```

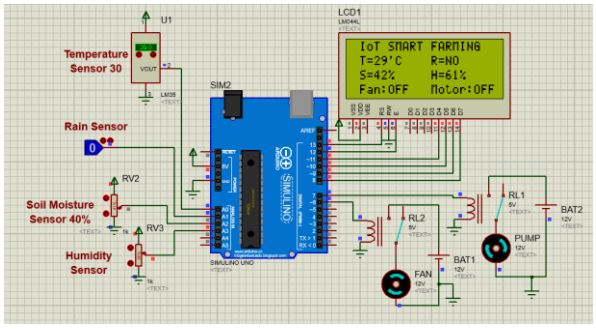
```
    lcd.setCursor(0,3);
    lcd.print(" Fan:ON  ");
  }
  else
  {
    digitalWrite(Fan_Pin,LOW);
    lcd.setCursor(0,3);
    lcd.print(" Fan:OFF ");
  }

  if(SM<40 && R==0)
  {
    digitalWrite(Pump_Pin,HIGH);
    lcd.setCursor(10,3);
    lcd.print(" Motor:ON  ");
  }
  else
  {
    digitalWrite(Pump_Pin,LOW);
    lcd.setCursor(10,3);
    lcd.print(" Motor:OFF  ");
  }
}
```

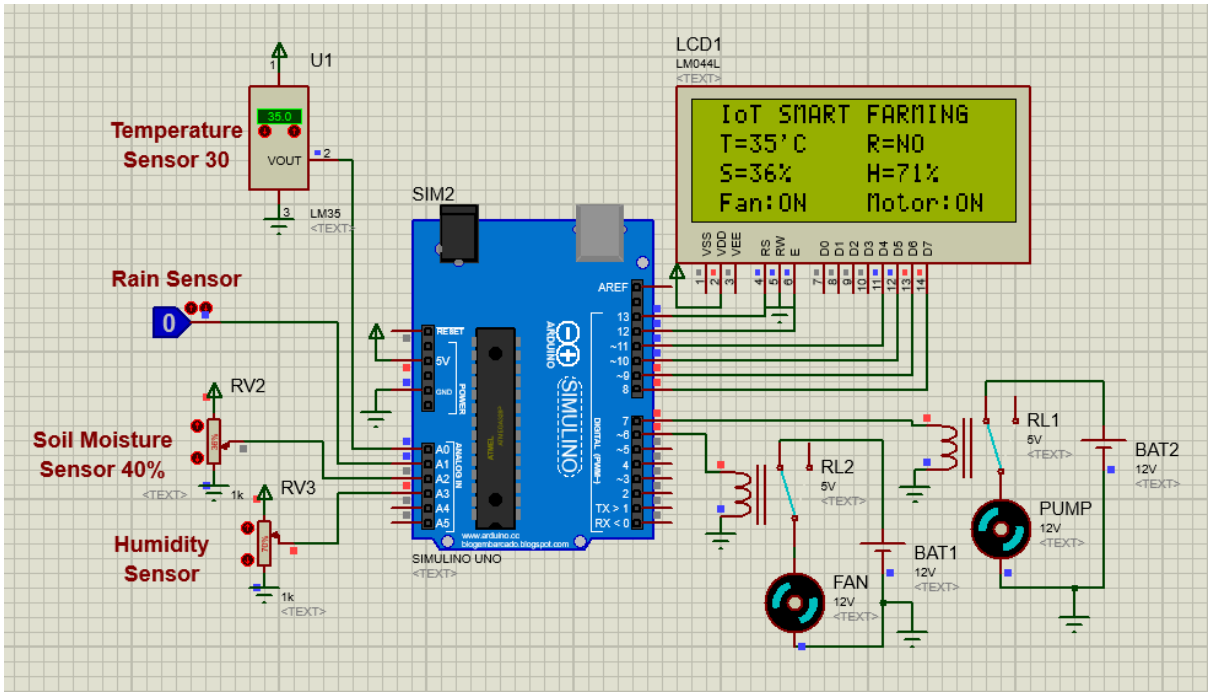
OUTPUT









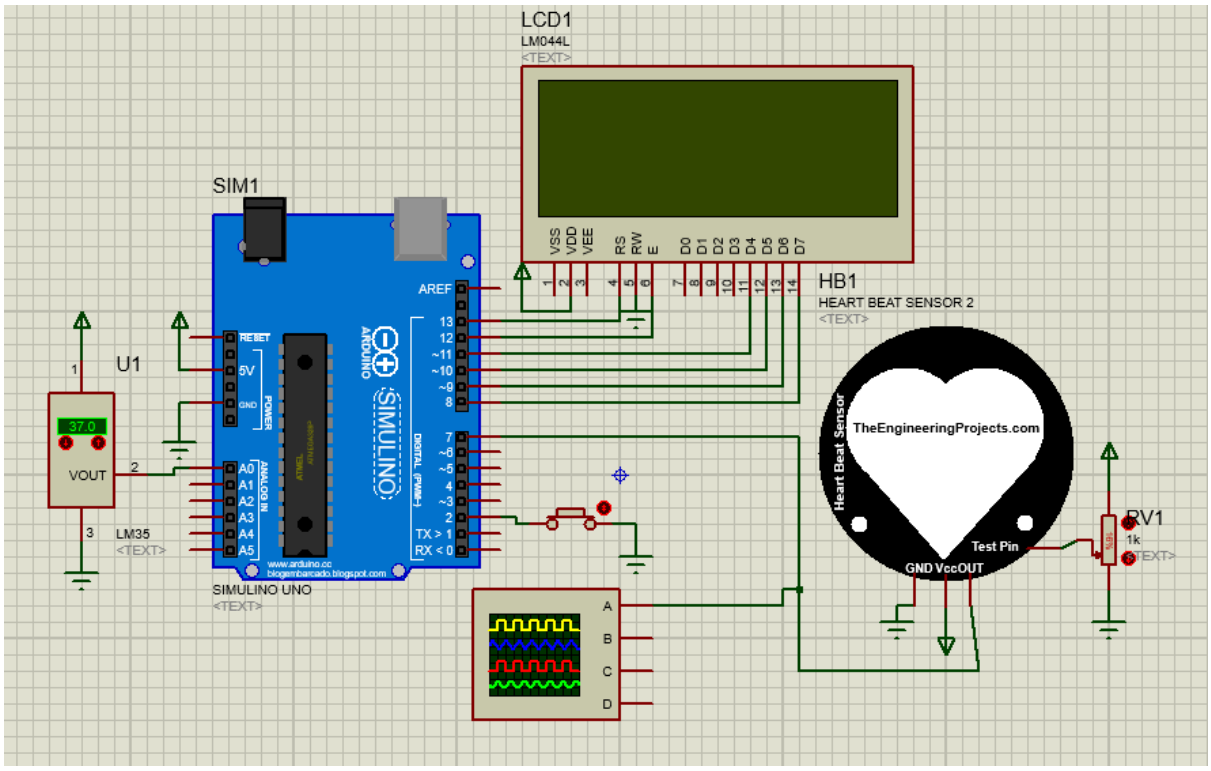

U1
Temperature
Sensor 30
35.0
VOUT
LM35
LCD1
LM044L
IoT SMART FARMING
T=35'C R=NO
S=36% H=71%
Fan:ON Motor:ON
VSS
VDD
VEE
RS
RW
E
D0
D1
D2
D3
D4
D5
D6
D7
SIM2
Rain Sensor
0
RV2
Soil Moisture
Sensor 40%
1k
RV3
Humidity
Sensor
1k
SIMULINO UNO
AREF
RESET
5V
GND
POWER
ANALOG IN
A0
A1
A2
A3
A4
A5
TX > 1
RX < 0
RL1
5V
RL2
5V
BAT2
12V
PUMP
12V
BAT1
12V
FAN
12V

10. Patient Health Monitoring System Using Arduino.

Requirements

1. Arduino Uno
2. LCD – LM044L
3. LM35
4. BUTTON
5. Heart Beat Sensor
6. Pot HG
7. Connecting wires

Circuit Connection



Instruction: **i.** Include the Timer one Library to Arduino IDE,

ii. Upload *HeartBeatSensorTEP2.Hex* to heart beat sensor.

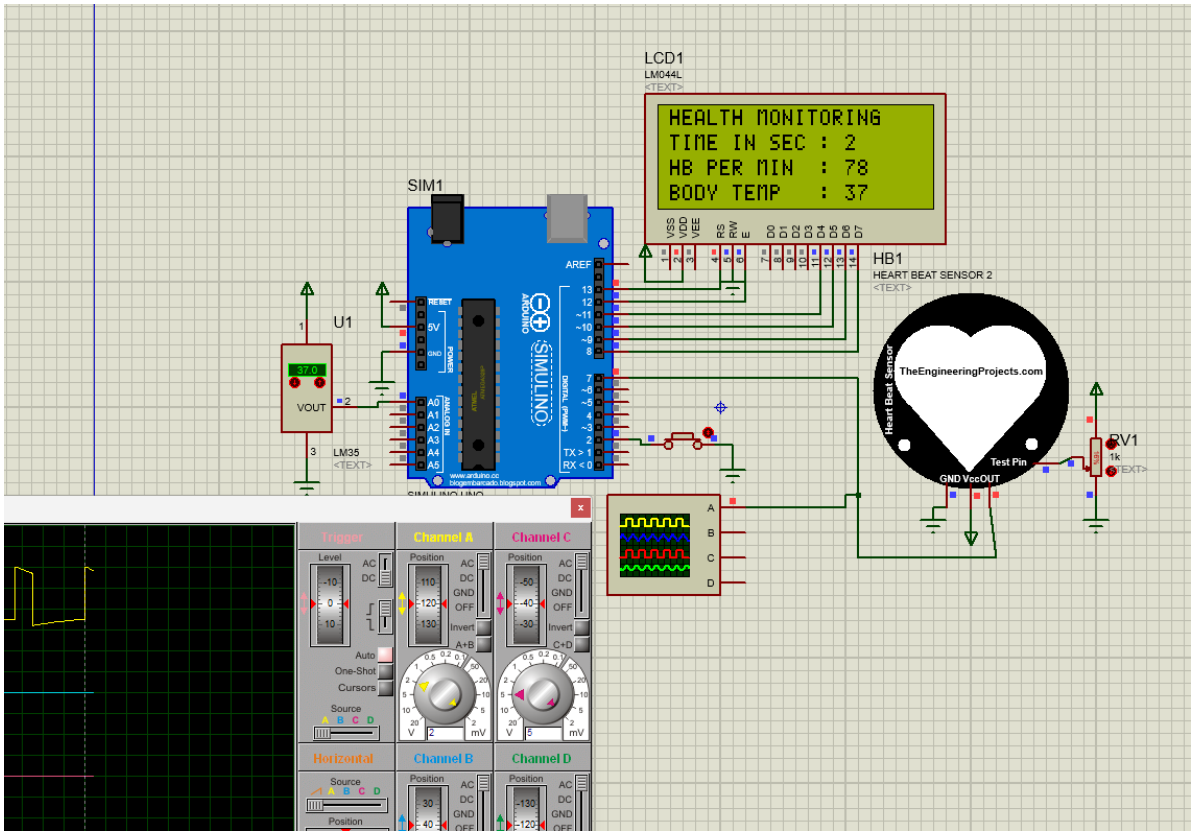
Program

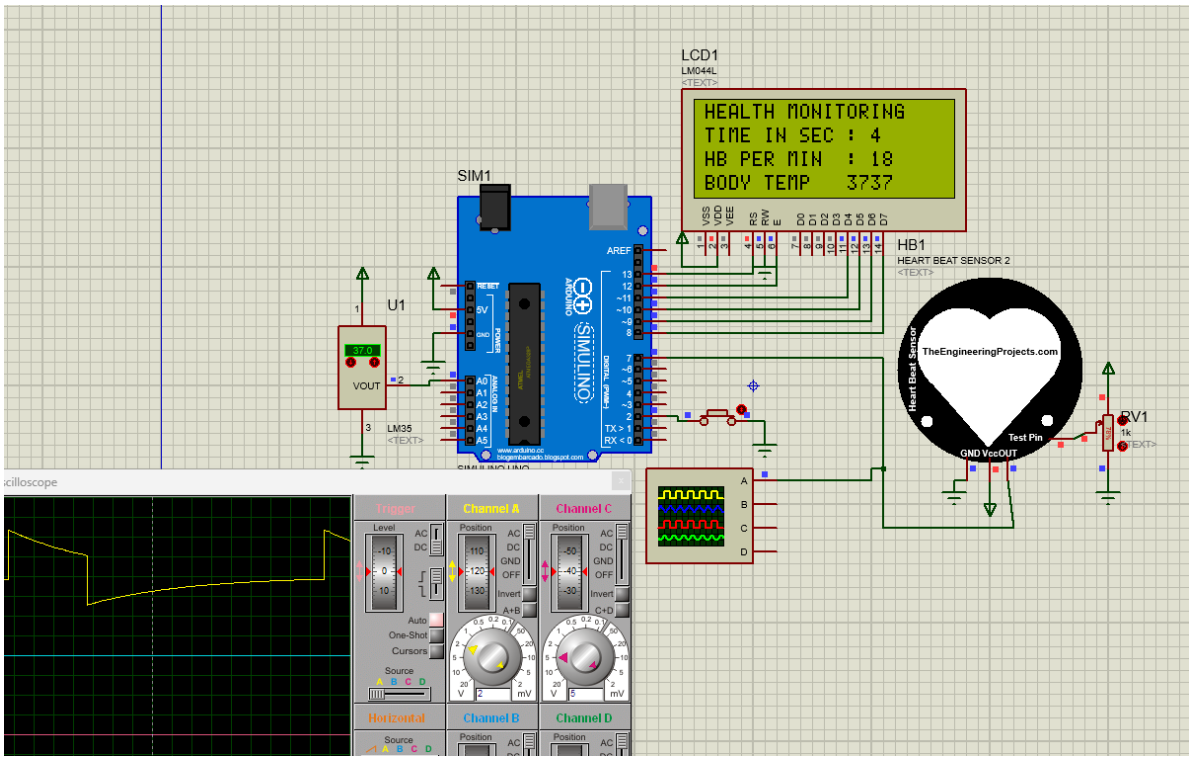
```
#include <TimerOne.h>

#include <LiuidCrystal.h>
LiuidCrystal lcd(13, 12, 11, 10, 9, 8);

int val;
int tempPin = A0;// temperature Sensor Pin
int HBSensor = 7;// Sensor Pin
int HBCount = 0;
int HBCheck = 0;
int TimeinSec = 0;
int HBperMin = 0;
```

```
int HBStart = 2;
int HBStartCheck = 0;

void setup()
{
  lcd.begin(20, 4);
  pinMode(HBSensor, INPUT);
  pinMode(HBStart, INPUT_PULLUP);
  Timer1.initialize(800000);
  Timer1.attachInterrupt( timerIsr );
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("HEALTH MONITORING");
  lcd.setCursor(0,1);
  lcd.print("TIME IN SEC : ");
  lcd.setCursor(0,3);
  lcd.print("BODY TEMP   : ");
  lcd.setCursor(0,2);
  lcd.print("HB PER MIN  : ");
 }
void loop()
{
  val = analogRead(tempPin);
  float mv = (val/1023.0)*5000;
  int cel = mv/10;
  lcd.setCursor(14,3);
  lcd.print(cel);
  lcd.print("   ");
  delay(100);
 if(digitalRead(HBStart) == LOW)
  {
    HBStartCheck = 1;
  }

  if(HBStartCheck == 1)
  {
    if((digitalRead(HBSensor) == HIGH) && (HBCheck == 0))
      {
        HBCount = HBCount + 1;
        HBCheck = 1;
      }
```

```
    if((digitalRead(HBSensor) == LOW) && (HBCheck == 1))
    {
      HBCheck = 0;
    }

    if(TimeinSec == 10)
    {
        HBperMin = HBCount * 6;;
        HBStartCheck = 0;
        lcd.setCursor(14,2);
        lcd.print(HBperMin);
        lcd.print(" ");
        HBCount = 0;
        TimeinSec = 0;
    }
  }
}

void timerIsr()
{
  if(HBStartCheck == 1)
  {
    TimeinSec = TimeinSec + 1;
    lcd.setCursor(14,1);
    lcd.print(TimeinSec);
    lcd.print(" ");
  }
}
```

OUTPUT


LCD1
LM044L
HEALTH MONITORING
TIME IN SEC : 2
HB PER MIN : 78
BODY TEMP : 37
SIM1
U1
VOUT
LM35
HB1
HEART BEAT SENSOR 2
TheEngineeringProjects.com
Heart Beat Sensor
Test Pin
GND VccOUT
RV1
1k
Trigger
Channel A
Channel C
Horizontal
Channel B
Channel D

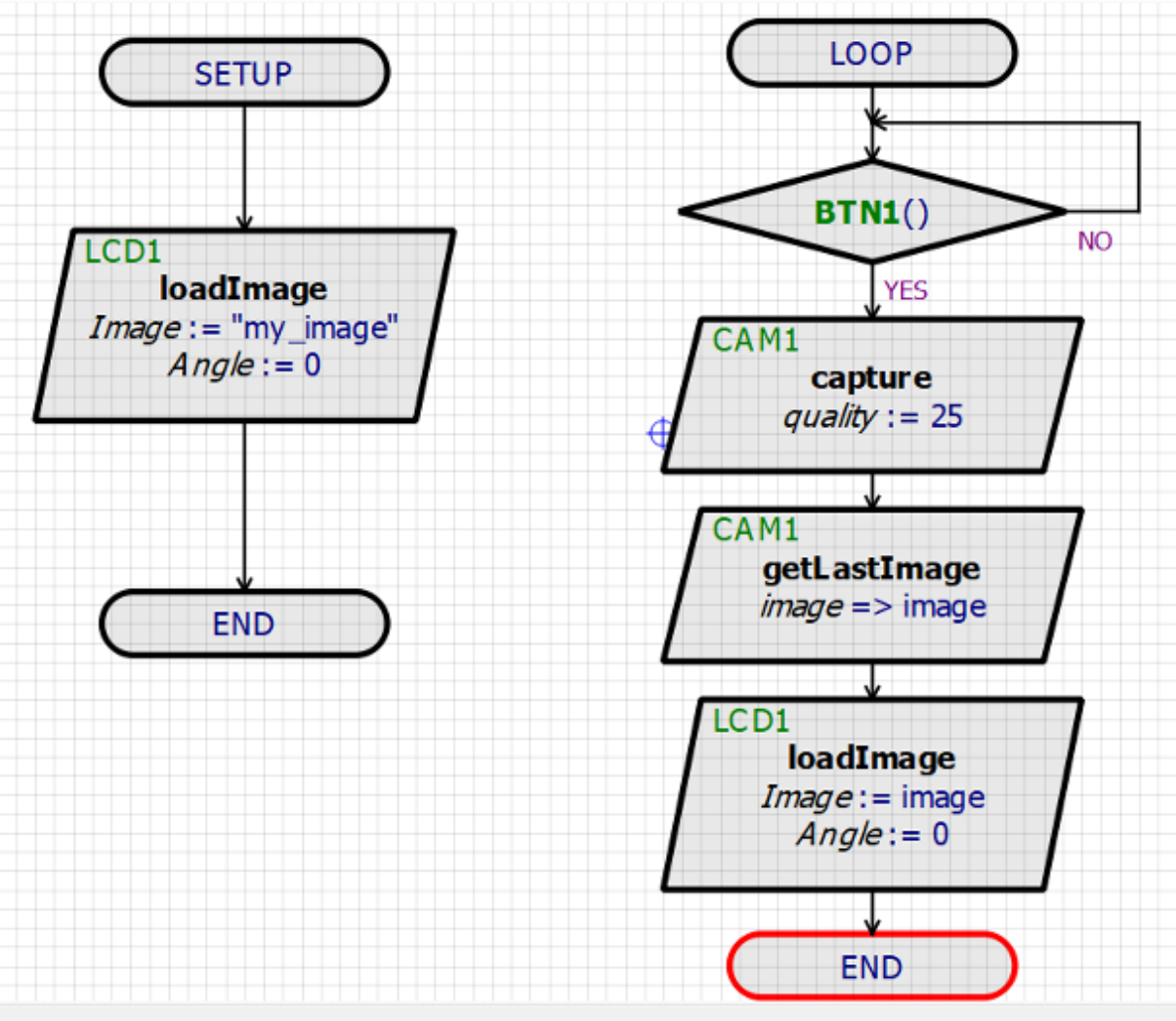

LCD1
LM044L
HEALTH MONITORING
TIME IN SEC : 4
HB PER MIN : 18
BODY TEMP 3737
SIM1
U1
VOUT
LM35
HB1
HEART BEAT SENSOR 2
TheEngineeringProjects.com
Heart Beat Sensor
Test Pin
GND VccOUT
RV1
1k
Oscilloscope
Trigger
Channel A
Channel C
Horizontal
Channel B
Channel D

11. Interfacing the regular USB webcam with the device and capture the image.

Requirements

1. Raspberry pi
2. Pi camera- picamera
3. Button- break through
4. LCD- TFT LCD
5. Image with extension of .png

Visual Designers



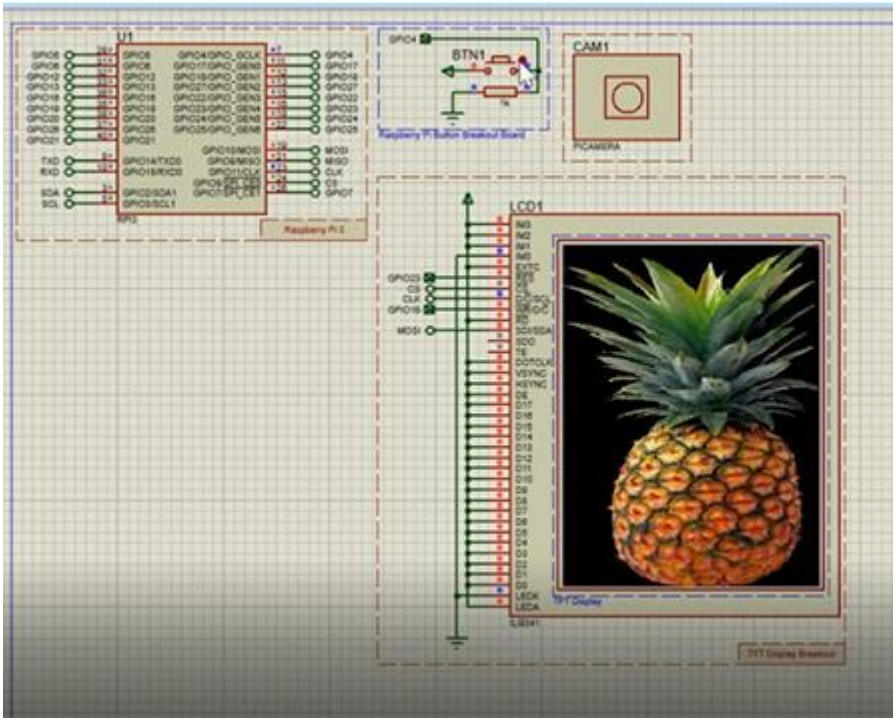
Note:

1. Take any image with extension of *.png keep it in the desktop
2. Add the components from left hand side peripherals – right click-> add peripherals

In category

a. select → Breakout Peripherals ---→ in that choose Momentary Action push Button click to add button
b. select → Camera-→ Raspberry Pi camera Module click to add button
c. Display →Displays ---→ select the TFT display click to add button Drag and drop the Function of the above module to the setup and loop

OUTPUT

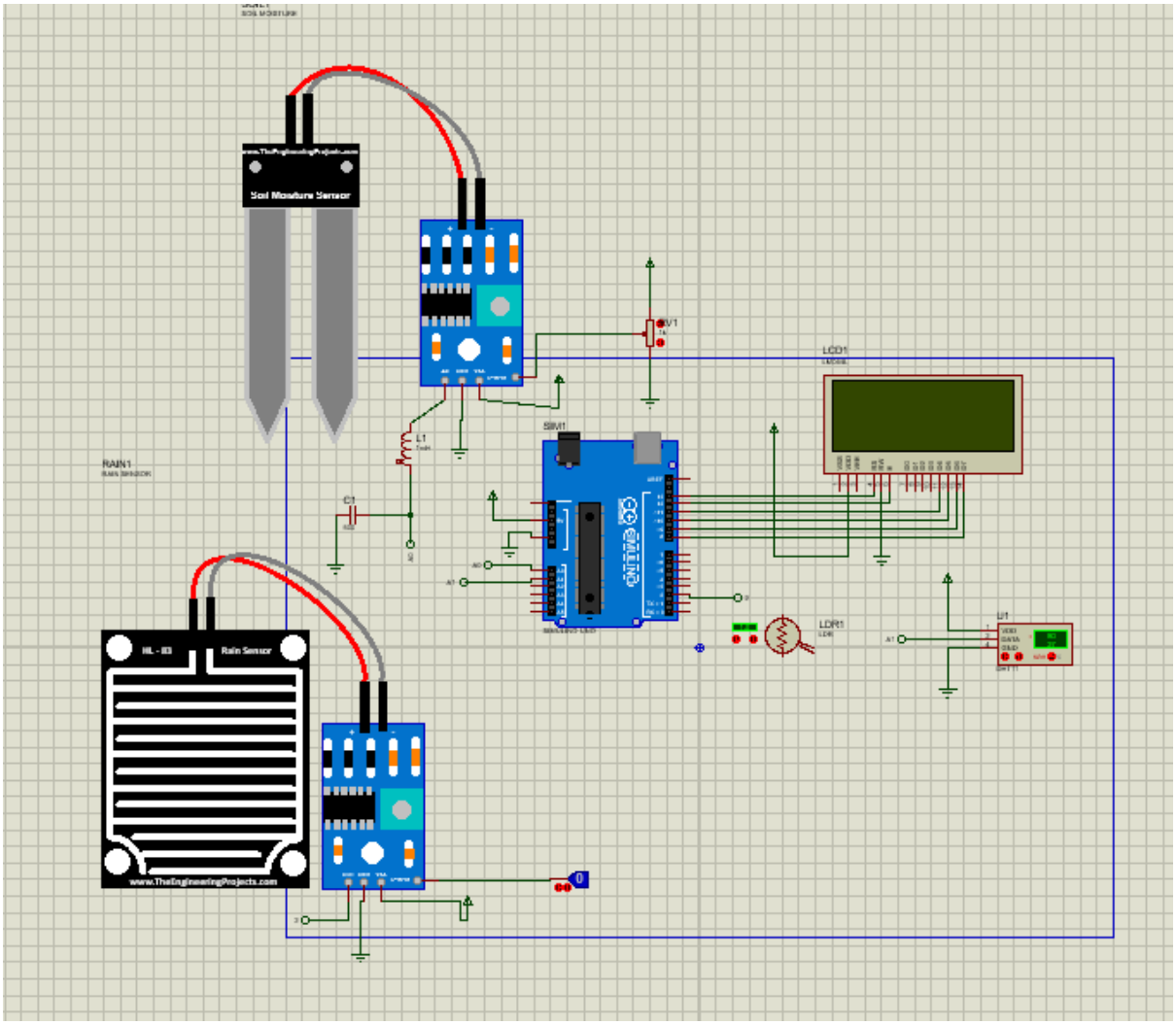


12. Build a circuit using Arduino based weather reporting over IOT

Requirements

1. Arduino Uno
2. Soil Moisture sensor
3. Rain sensor
4. DHT11
5. LDR-LDR – Generic model
6. LCD- LM044L (20x4 Alphanumeric lcd)
7. Capacitor -CAP
8. Potentiometer - POT-HG
9. Inductor – Inductor Primitive
10. Logicstate – Active
11. Connecting wires

Circuit Connection



Note:

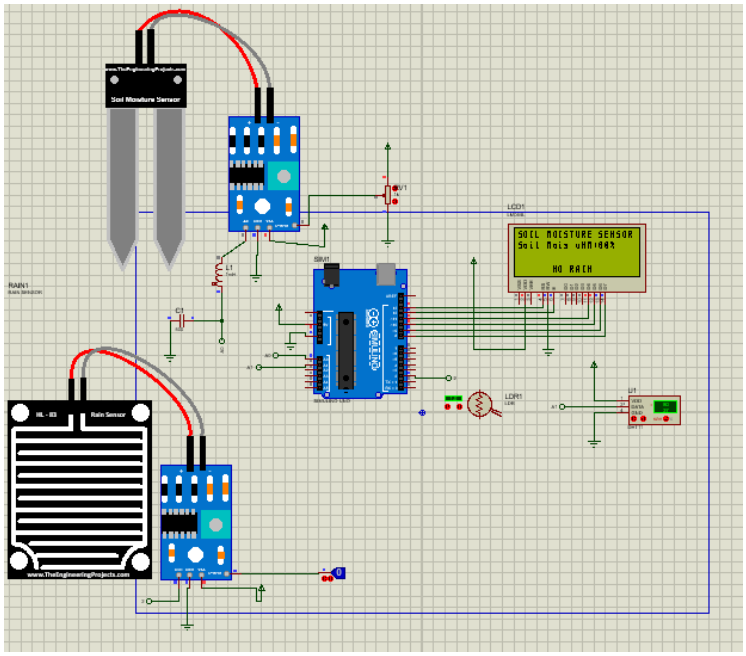
1. Add library files for the soil and rain sensors – by double click on the sensor upload the *.hex library of respective sensors
2. In Arduino IDE Upload DHT Library

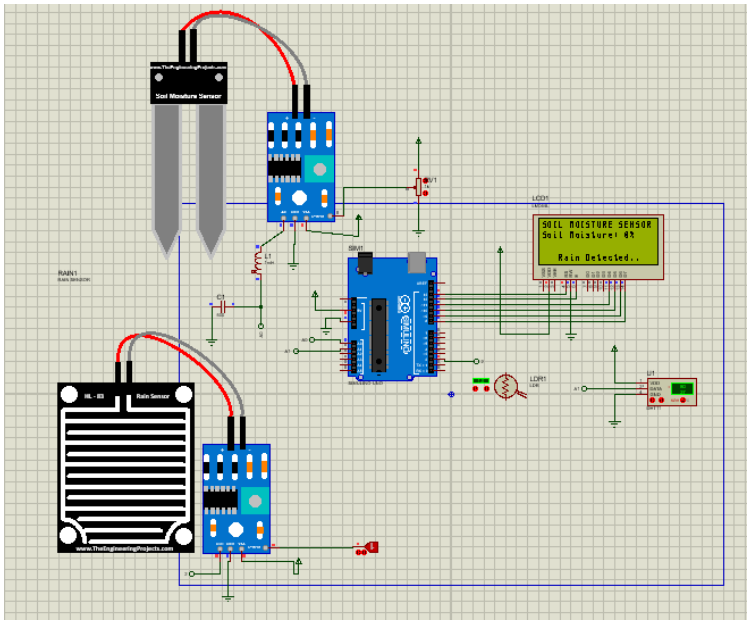
Program:

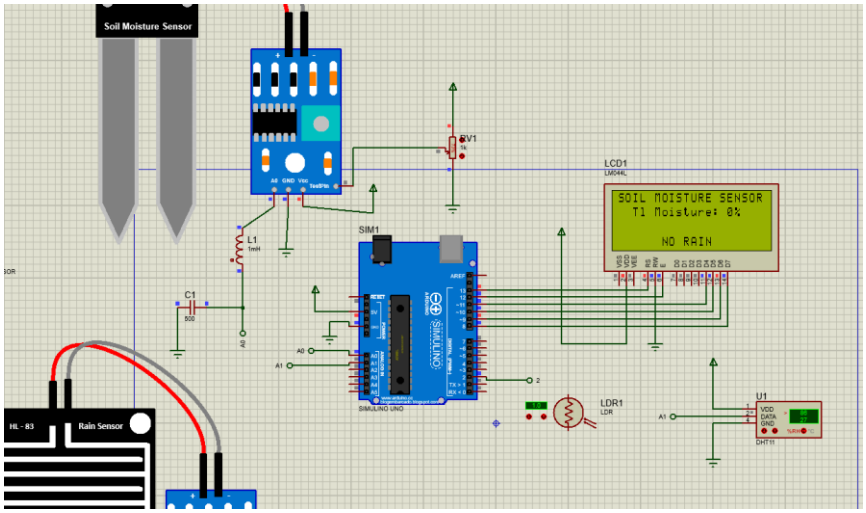
```
// include the library code:
#include <LiuidCrystal.h> //library for LCD
// initialize the library with the numbers of the interface pins
LiuidCrystal lcd(13, 12, 11, 10, 9, 8);
#include "dht.h"
#define dht_apin A1
dht DHT;
// defines pins numbers
const int SensorPin = A0;
const int SENSOR_PIN = 2;
void setup()
{
}
pinMode (SENSOR_PIN, INPUT);
lcd.begin(20, 4); // set up the LCD's number of columns and rows:
lcd.setCursor(0,0);
lcd.print("Weather Report");
void loop()
{
int Val = analogRead(SensorPin);
int Sensor_Val = digitalRead(SENSOR_PIN);
// Prints Message on the LCD
int Moisture = map(Val,0,1023,0,100);
DHT.read11(dht_apin);
lcd.setCursor(0,1);
lcd.print("Soil Moisture: ");
lcd.print(Moisture);
lcd.print("%   ");
if (Sensor_Val == HIGH) //If Sensor Detected the Rain
{
//digitalWrite(RLED_PIN, HIGH);
//digitalWrite(GLED_PIN, LOW);

    lcd.setCursor(0, 3);
    lcd.print("   Rain Detected              ");
    delay(100);
    lcd.setCursor(0, 3);
```

```
    lcd.print("   Rain Detected.            ");
    delay(100);
    lcd.setCursor(0, 3);
    lcd.print("   Rain Detected..            ");
    delay(100);
    lcd.setCursor(0, 3);
    lcd.print("   Rain Detected...            ");
    delay(100);
  }
  else
  {
    lcd.setCursor(0, 3);
    lcd.print("      NO RAIN            ");
    //digitalWrite(RLED_PIN, LOW);
    //digitalWrite(GLED_PIN, HIGH);
  }


int Temp = DHT.temperature;
  lcd.setCursor(0,1);
  lcd.print("  TM:");
  lcd.print(Temp);
  lcd.print("C  ");

  //Humidity Sennsing
  int Humidity = DHT.humidity;
  lcd.setCursor(11,1);
  lcd.print("HM:");
  lcd.print(Humidity);
  lcd.print("%  ");
}
```

OUTPUT


Soil Moisture Sensor
SOIL MOISTURE SENSOR
Soil Mois uMN100%
NO RAIN
Rain Sensor


Soil Moisture Sensor
SOIL MOISTURE SENSOR
Soil Moisture: 4%
Rain Detected..
Rain Sensor


Soil Moisture Sensor
LCD1
SOIL MOISTURE SENSOR
T1 Moisture: 0%
NO RAIN
SIM1
LDR1
U1
Rain Sensor

Viva – Voce uestion and Answer

1. What is Arduino?
   Arduino is an open-source microcontroller platform used for building embedded and IoT applications.

2. Difference between microcontroller and microprocessor?
   A microcontroller has CPU, memory, and I/O on a single chip, while a microprocessor needs external components.

3. Which microcontroller is used in Arduino UNO?
   ATmega328P.

4. Operating voltage of Arduino UNO?
   5V.

5. What are digital and analog pins?
   Digital pins read/write HIGH or LOW, analog pins read varying voltage values.

6. Difference between `digitalWrite()` and `analogWrite()`?
   `digitalWrite()` outputs HIGH/LOW, `analogWrite()` outputs PWM signals.

7. Why is PWM called analogWrite?
   Because it simulates analog output using digital pulses.

8. Range of `analogRead()`?
   0 to 1023.

9. Range of `analogWrite()`?
   0 to 255.

10. What is PWM?
    Pulse Width Modulation controls power by varying ON–OFF time.

11. Why use a resistor with LED?
    To limit current and protect the LED.

12. Purpose of `pinMode()`?
    To set a pin as INPUT or OUTPUT.

13. Difference between `delay()` and `millis()`?
    `delay()` blocks execution, `millis()` is non-blocking.

14. What is a potentiometer?
A variable resistor used to vary voltage.

15. What is baud rate?
Speed of serial communication.

16. Use of `Serial.begin()`?
Initializes serial communication.

17. Use of `setup()` and `loop()`?
`setup()` runs once, `loop()` runs continuously.

18. Wrong baud rate effect?
Data appears as garbage characters.

19. What is an interrupt?
A signal that pauses normal execution to run a special routine.

20. Applications of Arduino?
Robotics, IoT, automation, sensor monitoring.

B. Raspberry Pi –

1. What is Raspberry Pi?
A low-cost single-board computer.

2. Arduino vs Raspberry Pi?
Arduino is a microcontroller, Raspberry Pi is a microprocessor-based system.

3. Microcontroller or microprocessor?
Microprocessor.

4. Operating system used?
Raspberry Pi OS (Linux-based).

5. What is GPIO?
General Purpose Input Output pins.

6. Number of GPIO pins?
40-pin header (varies by model).

7. Default programming language?
   Python.

8. GPIO difference between Arduino and Pi?
   Arduino GPIO works at 5V, Pi GPIO works at 3.3V.

9. GPIO voltage level of Pi?
   3.3V.

10. Why level shifting is reuired?
    To protect Pi GPIO from 5V devices.

11. Use of SD card?
    Stores OS and programs.

12. Can Pi work without OS?
    No, it reuires an OS.

13. What is SSH?
    Remote terminal access.

14. What is VNC?
    Remote graphical desktop access.

15. Use of HDMI?
    Display output.

16. I2C, SPI, UART?
    Communication protocols.

17. How to interface LED?
    Using GPIO pin and resistor.

18. Role of Python?
    Control hardware and develop applications.

19. Applications of Raspberry Pi?
    Media center, IoT, servers, AI projects.

20. Why called single board computer?
    Entire computer is on one board.

C. Simulation (Proteus)

1. What is simulation?
   Virtual testing of circuits before hardware implementation.

2. Why use simulation?
   To reduce cost and avoid hardware damage.

3. What is Proteus?
   An electronic circuit simulation software.

4. Advantages of Proteus?
   Easy debugging, saves time and cost.

5. Can Arduino be simulated?
   Yes.

6. File reuired in Proteus?
   HEX file.

7. What is HEX file?
   Compiled machine code of Arduino program.

8. Why LED glows faintly in Proteus?
   Due to PWM and simulation limitations.

9. Simulation vs real hardware?
   Simulation is virtual; hardware is real-time.

10. Limitations of Proteus?
    Not 100% accurate.

11. Can Raspberry Pi be simulated?
    Limited support.

12. Components tested in simulation?
    LEDs, motors, sensors, controllers.

13. Virtual terminal?
    Used to view serial output.

14. Why resistor is important?
    To limit current.

15. Wrong connections effect?
    Circuit will not work.

16. Is simulation fully accurate?
    No.

17. What is debugging?
    Finding and fixing errors.

18. PWM behavior in Proteus?
    Low PWM may still show LED glow.

19. Advantages in labs?
    Safe learning environment.

20. When prefer hardware?
    For real-time testing.

D. Comparison

1. Arduino vs Raspberry Pi
   Arduino is for control tasks; Raspberry Pi is for computing tasks.

2. Prefer Arduino when?
   Real-time control applications.

3. Prefer Raspberry Pi when?
   OS-based applications.

4. Real-time vs General-purpose system
   Real-time responds instantly; general-purpose does not guarantee timing.

5. Role of simulation in IoT
   Helps test designs before deployment.

1. What is a temperature sensor?
A temperature sensor measures the temperature of the surrounding environment.

2. Which temperature sensor is commonly used with Arduino?
LM35 and DHT11/DHT22.

3. What is the output of LM35?
Analog voltage proportional to temperature.

4. What is the scale factor of LM35?
10 mV per degree Celsius.

5. Why is ADC required for LM35?
Because LM35 gives analog output.

6. What is a rain sensor?
A rain sensor detects rainfall or water droplets.

7. What is the working principle of a rain sensor?
Change in resistance due to water presence.

8. Is rain sensor analog or digital?
It can provide both analog and digital output.

9. Mention one application of rain sensor.
Automatic irrigation system.

10. What is a potentiometer?
A variable resistor used to control voltage.

11. How many terminals does a potentiometer have?
Three terminals.

12. Why is potentiometer connected to analog pin?
Because it provides varying voltage.

13. Give one application of potentiometer.
LED brightness control.

14. What is a relay?
A relay is an electrically operated switch.

15. Why do we use a relay with Arduino?
To control high-voltage devices safely.

16. What is electrical isolation?
Separation between control and load circuits.

17. Give one application of relay.
Home automation.

18. What is a camera module?
A device used to capture images and videos.

19. Which camera is used with Raspberry Pi?
Raspberry Pi Camera Module.

20. What interface does Pi camera use?
CSI interface.

21. Mention one application of camera module.
Surveillance system.

22. What is a Wi-Fi module?
A module that provides wireless internet connectivity.

23. Name a commonly used Wi-Fi module.
ESP8266 or ESP32.

24. Why Wi-Fi module is important in IoT?
For remote data transmission.

25. What protocol is used in Wi-Fi communication?
TCP/IP.

26. What is an LCD?
Liquid Crystal Display used to show data.

27. What is a 16×2 LCD?
It displays 16 characters in 2 rows.

28. Why do we use LCD in projects?
To display sensor values and messages.

29. What is the advantage of I2C LCD?
Uses fewer pins.

30. What is a resistor?
A component that limits current flow.

31. Why is resistor used with LED?
To protect LED from excess current.

32. What happens if resistor is not used?
LED may burn.

33. What are connecting wires?
Wires used to connect electronic components.

34. Name types of jumper wires.
Male-to-Male, Male-to-Female, Female-to-Female.

35. Why are good connections important?
To ensure proper signal and power flow.

36. What is a sensor?
A device that converts physical parameters into electrical signals.

37. Difference between analog and digital sensors?
Analog gives continuous output, digital gives discrete output.

38. Why sensors are important in IoT?
They collect real-world data.

39. Why do LEDs glow slightly in Proteus?
Due to PWM and simulation limitations.

40. Why simulation is preferred before hardware?
To avoid hardware damage and reduce cost.

41. Explain how sensors, controller, and communication module work together in an IoT system.
Sensors collect data, microcontroller processes it, and Wi-Fi module sends it to the cloud.

41. What is an ultrasonic sensor?
An ultrasonic sensor measures distance using ultrasonic sound waves.

42. What is the working principle of ultrasonic sensor?
It sends a sound pulse and measures the time taken for the echo to return.

43. Which ultrasonic sensor is commonly used with Arduino?
HC-SR04.

44. What are the pins of HC-SR04?
Vcc, Trigger, Echo, and GND.

45. What is the formula to calculate distance?
Distance = (Time × Speed of Sound) / 2.

46. What is the speed of sound used in calculations?
Approximately 340 m/s.

47. Give one application of ultrasonic sensor.
Obstacle detection in robots.

48. What is an LED?
A semiconductor device that emits light when current flows through it.

49. Why LED is called a diode?
Because it allows current to flow in one direction.

50. Why do we use a resistor with LED?
To limit current and prevent damage.

51. What happens if LED is connected in reverse bias?
It will not glow.

52. Give one application of LED.
Status indication.

53. What is an RGB LED?
An LED that combines Red, Green, and Blue LEDs in one package.

54. How many pins does an RGB LED have?
4 pins (R, G, B, Common).

55. What is common anode and common cathode?
Common terminal connected to Vcc (anode) or GND (cathode).

56. How are different colors produced in RGB LED?
By varying intensity of Red, Green, and Blue.

57. Why PWM is used with RGB LED?
To control brightness and color mixing.

58. What is a traffic light system?
A system that controls vehicle movement using signal lights.

59. Which colors are used in traffic lights?
Red, Yellow, and Green.

60. What does each color indicate?
Red – Stop, Yellow – Ready, Green – Go.

61. How is traffic light implemented using Arduino?
By controlling LEDs with timed delays.

62. What Arduino function is used for timing?
delay() or millis().

63. Give one application of traffic light project.
Smart city traffic management.

64. What is an actuator?
A device that converts electrical signals into physical action.

65. Is LED a sensor or actuator?
Actuator.

66. Difference between sensor and actuator?
Sensor collects data; actuator performs action.

67. Difference between ultrasonic and IR sensor?
Ultrasonic uses sound waves; IR uses infrared light.

68. Which is better for distance measurement?
Ultrasonic sensor.

69. Why ultrasonic sensor may give inaccurate readings in simulation?
Due to timing and echo simulation limitations.

70. Why PWM pins are required for RGB LED?
To vary brightness of each color.

71. Explain the role of ultrasonic sensor and LED in obstacle detection system.
Ultrasonic sensor detects distance and LED gives visual indication.